

SIMULACRO PARCIAL PRÁCTICO

APLICACIÓN WEB EN JSP

Inmobiliaria web

JSP + JSPF + JDBC + MySQL 8 + Bootstrap 5

Elemento	Detalle
Asignatura	Programación web con Java (Java JSP y JSPF)
Tipo de práctica	Taller guiado paso a paso, proyecto completo
Arquitectura	Modelo 1: Páginas JSP + Fragmentos reutilizables .jspf
Base de datos	MySQL 8
Conexión	Archivo conexion.jspf con JDBC y PreparedStatement
Interfaz	Bootstrap 5.3 desde CDN, diseño responsivo
Servidor	Apache Tomcat 9
Archivos del proyecto	
Duración estimada	3 Sprints de 1 semana cada uno (Aprox. 3 semanas)
Estudiantes	Jhon Alexander Granados Duque Lewing Santiago Mendez Trilloz

Contenido

1. Presentación del taller	5
1.2 Competencias que se practican	5
1.3 Entregables	5
1.4 Plan de trabajo por sprints.....	6
2. Herramientas y requisitos.....	6
3. La base de datos del proyecto.....	7
3.1 Las restricciones UNIQUE que se notan en la aplicación	8
3.2 Modelo Entidad-Relación	9
4. Arquitectura de la aplicación	9
4.1 Qué es un archivo .jspf y por qué se usa	10
4.2 Archivos del proyecto.....	10
4.3 Recorrido de una petición	11
Paso 1. Crear el proyecto y la estructura de carpetas	12
Paso 2. Instalar el driver JDBC de MySQL.....	13
3. Crear la base de datos y los usuarios de prueba	13
Paso 3.1 Usuarios con los que inicia sesión	14
Paso 3.2 Verificación	14
Paso 4. El archivo de conexión: conexion.jspf	15
Paso 4.1 Explicación parte por parte	16
Paso 4.2 La URL de conexión, parámetro por parámetro	16
Paso 4.3 Probar la conexión antes de seguir.....	17
Paso 5. Métodos de apoyo: Utilidades.jspf	17
Paso 5.1 Para qué sirve cada método	19
Paso 6. La plantilla visual con Bootstrap 5	19
Paso 6.1 Cabecera.jspf.....	19
Paso 6.2 Qué aporta Bootstrap 5 aquí.....	20
Paso 6.3 pie.jspf.....	21

Paso 6.4 estilos.css	22
Paso 7. Control de acceso: seguridad.jspf	22
Paso 7.1 Cómo se usa en cada página	23
Paso 8. Inicio y cierre de sesión	24
Paso 8.1 login.jsp	25
Paso 8.2 acceso.jsp	26
Paso 8.3 Los cuatro puntos clave de la validación	27
Paso 8.4 logout.jsp	27
Paso 8.5 Registro público: registro.jsp y registrar.jsp	28
Paso 8.6 index.jsp	29
Paso 8.7 landing.jsp	30
Paso 9. El panel principal: inicio.jsp	31
Paso 9.1 Detalles que vale la pena mirar	33
Paso 10. Módulo de propiedades	33
Paso 10.1 listar.jsp: El catálogo con filtros	34
Paso 10.2 nueva.jsp: Publicar una propiedad	36
Paso 10.3 detalle.jsp: La ficha de la propiedad	37
Bloque 1 de 4: Cargar los datos de la propiedad	38
Bloque 2 de 4: La galería de imágenes	40
Bloque 3 de 4: Características (Resuelve la relación N:M)	41
Bloque 4 de 4: El panel de acciones (Cambia según el rol)	42
Paso 10.4 editar.jsp: Edición de una propiedad	43
Paso 10.5 acciones.jsp: El controlador de propiedades	44
Estructura general (Preparación y doble validación de rol)	44
Acción crear (con transacción propiedad + características)	45
Acción editar	46
Acciones baja y favorito	47
Cierre: Manejo de la restricción UNIQUE	48

Paso 11. Módulo de citas	49
Paso 11.1 cita/nueva.jsp	49
Paso 11.2 cita/acciones.jsp	50
Paso 11.3 cita/mis-citas.jsp.....	51
Paso 11.4 cita/agenda.jsp.....	51
Paso 12. Módulo de solicitudes	52
Paso 12.1 solicitud/nueva.jsp	52
Paso 12.2 solicitud/acciones.jsp	52
Paso 12.3 solicitud/mis-solicitudes.jsp	54
Paso 12.4 solicitud/gestión.jsp.....	54
Paso 13. Administración de usuarios	55
Paso 13.1 admin/usuarios.jsp	55
Paso 13.2 admin/acciones.jsp	56
Paso 14. Reportes	57
Paso 15. Desplegar y probar la aplicación	58
Paso 15.1 Puesta en marcha	58
Paso 15.2 Guion de prueba completa	58
Paso 15.3 Comprobación en la base de datos	59
16. Consultas obligatorias	60
17. Metodología Scrum (Resumen)	60
18. Pruebas unitarias	60
19. Errores frecuentes y cómo resolverlos	61
20. Conclusiones	62
Anexo A. Inventario de archivos entregados	62

1. Presentación del taller

Durante este proyecto se desarrolla, de principio a fin, la aplicación web con la que una inmobiliaria ficticia gestiona sus propiedades. Un visitante consulta el catálogo público; al registrarse se convierte en cliente y puede buscar propiedades, marcarlas como favoritas, agendar una cita para visitarlas y radicar una solicitud de compra o arriendo junto con sus documentos. La inmobiliaria (agente) publica y administra sus propias propiedades, atiende las citas agendadas sobre ellas, y aprueba o rechaza las solicitudes recibidas. El administrador tiene acceso total: gestiona los usuarios y sus roles, y consulta reportes.

Todo el proyecto se escribe con páginas JSP y fragmentos .jspx. La conexión a MySQL se centraliza en un único archivo, conexionjspx, y la presentación se resuelve con Bootstrap 5 para que la aplicación se vea bien tanto en computador como en celular.

1.2 Competencias que se practican

- Crear un proyecto web Java y desplegarlo en Apache Tomcat
- Separar código repetido en fragmentos .jspx e incluirlos con la directiva include.
- Conectar una aplicación web a MySQL con JDBC y el driver mysql-connector-j.
- Escribir consultas seguras con PreparedStatement y evitar la inyección SQL.
- Manejar transacciones (commit/rollback) cuando una operación toca varias tablas.
- Controlar el acceso con sesiones HTTP y perfiles de usuario (visitante, cliente, inmobiliaria, administrador).
- Construir interfaces responsivas con el sistema de rejilla y los componentes de Bootstrap 5.
- Traducir los errores de las restricciones UNIQUE de la base de datos a mensajes claros para el usuario.

1.3 Entregables

#	Entregable	Descripción
1	Script de la base de datos	Archivo .sql ejecutado en MySQL con las 13 tablas, los datos de prueba (10 registros en tablas principales) y los usuarios de prueba.
2	Proyecto web	Repositorio con las páginas JSP, los 6 fragmentos .jspx, el CSS y el web.xml.
3	Modelos y diagramas	MER, modelo relacional, diccionarios de datos y diagrama de casos de uso

4	Documentación Scrum	Backlog y los 3 sprints (Planning/Review/Retrospective), con tablero de seguimiento
5	Evidencia de funcionamiento	Capturas de pantalla del recorrido completo por cada rol
6	Documento final	Explica la arquitectura, el modelo de datos, cada módulo, las consultas obligatorias y las pruebas unitarias

1.4 Plan de trabajo por sprints

Sprint	Días	Resultado esperado al terminar
1	1 – 7	Modelo de datos, conexión JDBC, Landing, registro, login con roles funcionando
2	8 – 14	CRUD de propiedades con imágenes/Características, filtros, perfil, favoritos
3	15 – 21	Citas, solicitudes, reportes, pruebas unitarias, documentación final

2. Herramientas y requisitos

Antes de empezar, verifique que tiene instalado y funcionando lo siguiente.

Herramienta	Versión usada	Para qué se usa	Cómo verificar
JDK	Java (JDK completo, no solo JRE)	Compilar el proyecto y correr las pruebas unitarias	<code>java -version</code> y <code>javac -version</code> en la consola
Apache Tomcat	8.5.96 (Bajo XAMPP)	Servidor de aplicaciones que ejecuta el proyecto	Abrir <code>http://localhost:8080</code>
MySQL	8.0.46	Motor de la base de datos	<code>mysql -u root -p</code> , o conectar desde MySQL Workbench
MySQL Workbench	-	Ejecutar el script y revisar las tablas	Conexión local a 127.0.0.1:3306
Driver JDBC	mysql-connector-j 26.7.0	Permite que Java hable con MySQL	Debe quedar en <code>WEB-INF/lib</code>
IDE	Visual Studio Code	Editar el proyecto (Estructura manual,	-

		sin Eclipse/NetBeans)	
Navegador	Chrome/Edge	Probar la aplicación	F12 para ver errores
Bootstrap	5.3	Desde CDN (No se descarga)	Estilos y componentes de la interfaz
Git/GitHub	-	Control de versiones, repositorio público	git --version

Importante: En este proyecto la base de datos corre en una instancia de MySQL 8 independiente (no la que trae XAMPP), en el puerto estándar 3306. Si usted usa la instancia de MySQL de XAMPP, revise el puerto y ajuste DB_URL en `conexion.jspf` en consecuencia.

Consejo: Si su instalación de Tomcat es la 10 u 11, tenga en cuenta que usan el espacio de nombres `jakarta.servlet` en vez de `javax.servlet` en el `web.xml` — aunque este proyecto no usa Servlets directamente, esto puede afectar la configuración del `web.xml` si la agrega.

3. La base de datos del proyecto

La aplicación trabaja sobre el esquema `inmobiliaria_simulacro`, con 13 tablas. Conviene tener presente que guarda cada tabla, porque cada módulo de la aplicación consulta o actualiza una parte concreta del modelo.

Tabla	Qué guarda	Módulo de la aplicación que usa
rol	Los cuatro perfiles: Administrador, inmobiliaria, cliente (y visitante, sin registro).	login (acceso.jsp)
usuario	Credenciales, correo, documento y estado de la cuenta	login, registro y control de sesión
perfil	Datos personales (nombres, apellidos, teléfono, dirección)	registro, edición de perfil
ciudad / tipo_propiedad	Catálogos usados por las propiedades	filtros, formularios de propiedades

propiedad	Encabezado de cada inmueble publicada	todos los modulos
imagen_propiedad	Galeria de fotos de cada propiedad	ficha de detalle
caracteristica/propiedad_caracteristica	Catálogo de características y su relación N:M con las propiedades	formulario y ficha de propiedad
cita	Citas agendadas sobre una propiedad	agenda del cliente y del agente
solicitud/documento_solicitud	Solicitudes de compra/arriendo y sus documentos radicados	módulo de solicitudes
Favorito	Propiedades marcadas por cada cliente	ficha de detalle

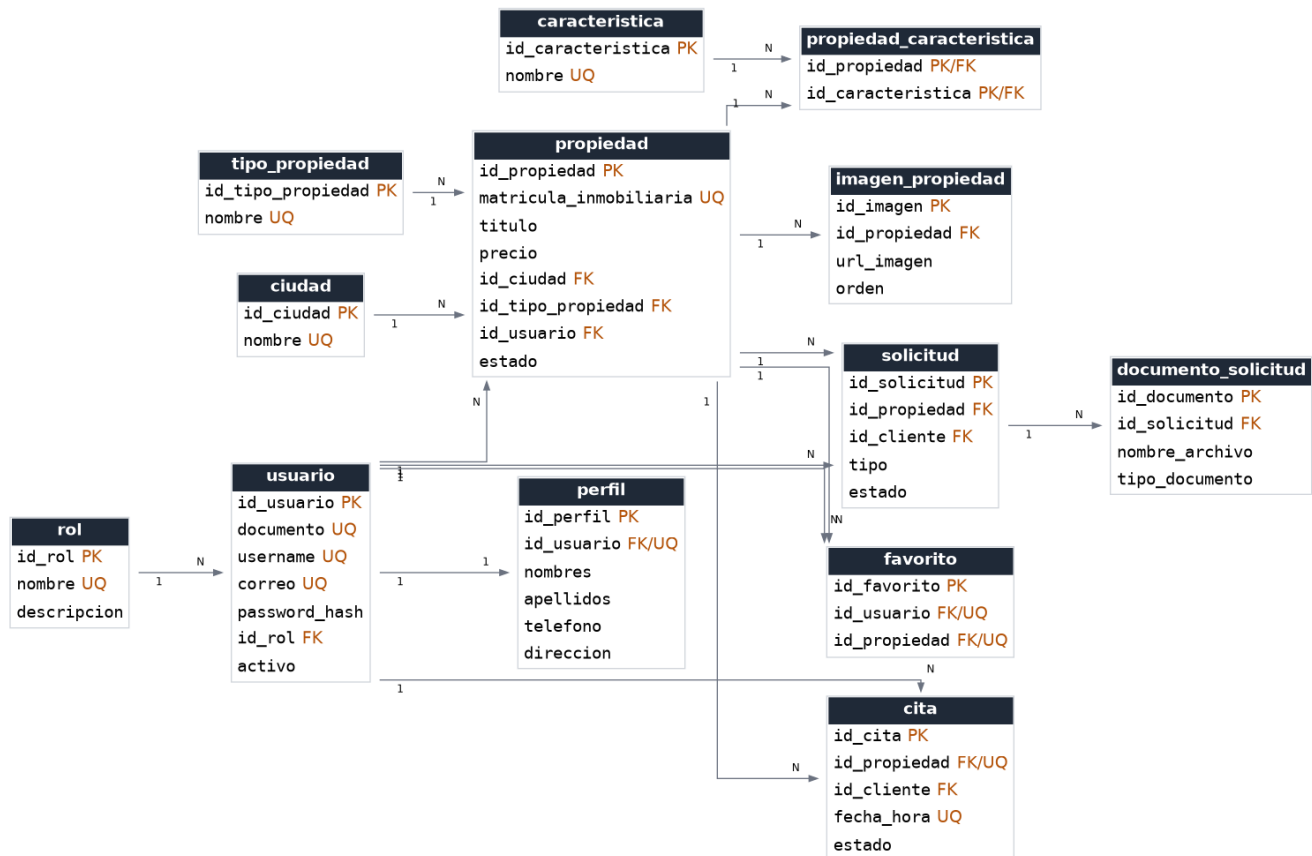
3.1 Las restricciones UNIQUE que se notan en la aplicación

El diseño de la base de datos no es decorativo: varias restricciones aparecen directamente en el comportamiento del programa y en los mensajes que ve el usuario.

Restricción	Qué impide	Dónde se nota en el código
uq_usuario_correo/uq_usuario_username	Dos cuentas con el mismo correo o usarlo	registrar.jsp muestra 'Ese correo ya está registrado'.
uq_propiedad_matricula	Dos propiedades con la misma matrícula inmobiliaria	propiedad/acciones.jsp muestra 'Ya existe una propiedad con esa matrícula inmobiliaria'.
uq_cita_propiedad_fecha	Dos citas en la misma propiedad al mismo horario	cita/acciones.jsp muestra 'Esa propiedad ya tiene una cita agendada en ese horario'.
uq_favorito_usuario_propiedad	Marcar la misma propiedad como favorita dos veces	Se resuelve con lógica de 'toggle'.
uq_perfil_usuario	Que un usuario tenga dos perfiles (Garantiza la relación 1:1)	Restricción a nivel de base de datos, no requiere manejo especial en el código

El diagrama de la página siguiente resume el modelo completo

3.2 Modelo Entidad-Relación



4. Arquitectura de la aplicación

El proyecto usa el llamado Modelo 1: la propia página JSP recibe la petición del navegador, consulta la base de datos y devuelve el HTML. No se escriben servlets ni clases Java aparte (confirmado con el docente: el tema de Servlets aún no se ha visto en clase); el código repetido se saca a fragmentos .jspf y así se conserva el orden.

4.1 Qué es un archivo .jspxf y por qué se usa

Un `.jspxf` (JSP Fragment) es un pedazo de página que no se ejecuta por sí solo: se pega dentro de otra página en el momento de compilar, mediante la directiva de inclusión estática:

```
<%@ include file="/WEB-INF/jspf/conexion.jsxpf" %>
```

Como la unión ocurre antes de compilar, la página final es una sola clase Java. Eso tiene dos consecuencias prácticas:

- Las variables y los métodos declarados en el fragmento quedan disponibles en la página que lo incluye (por eso `conexion.jsxpf` puede ofrecer el método `abrirConexion()`).
- Un mismo fragmento no se puede incluir dos veces en la misma página: se duplicarían las declaraciones y el compilador daría error.

4.2 Archivos del proyecto

Archivo	Rol	Qué hace
index.jsp	Entrada	Redirige a la Landing o al panel según haya sesión
landing.jsp	Vista pública	Buscador rápido y propiedades destacadas
login.jsp/acceso.jsp/logout.jsp	Autenticación	Formulario, validador y cierre de sesión
registro.jsp/registrar.jsp	Registro	Formulario y controlador de alta de usuarios (rol CLIENTE por defecto)
inicio.jsp	Vista	Panel principal con accesos según el rol
propiedad/*.jsp	CRUD	Listar(con filtros), nueva, editar, detalle, acciones (controlador).
cita/*.jsp	Módulo	Nueva, acciones (controlador), mis-citas (cliente), agenda (agenda/admin)
solicitud/*.jsp	Módulo	Nueva, acciones (controlador). mis-solicitudes (cliente), gestión (agente/admin)
admin/usuarios.jsp / admin/acciones.jsp	Administración	Asignar roles, activar/inactivar cuentas

reportes.jsp	Reportes	Las 5 consultas SQL obligatorias
WEB-INF/jspf/conexion.jspf	Fragmento	Conexión JDBC centralizada
WEB-INF/jspf/utilidades.jspf	Fragmento	Cifra, formato, validaciones, escape XSS
WEB-INF/jspf/seguridad.jspf	Fragmento	Control de acceso por sesión y rol (Páginas privadas)
WEB-INF/jspf/sesionOpcional.jspf	Fragmento	Lee la sesión sin redirigir (Páginas públicas).
WEB-INF/jspf/cabecera.jspf/pie.jspf	Fragmento	Plantilla visual común con menú dinámico por rol

4.3 Recorrido de una petición

Cuando un cliente hace clic en ‘Agendar cita’ ocurre, en este orden:

- El navegador envía un POST a `cita/acciones.jsp` con `accion=crear` y los datos del formulario.
- La página incluye `conexion.jspf` y `utilidades.jspf` (métodos disponibles), y `seguridad.jspf` (verifica que hay sesión y que el rol es CLIENTE).
- Abre una conexión, desactiva el autocommit y ejecuta el `INSERT` en la tabla `cita`.
- Si todo sale bien hace `commit`; si la restricción `UNIQUE` de horario falla, hace `rollback` y traduce el error a un mensaje claro.
- Cierra la conexión y responde con una redirección (`sendRedirect`) a "Mis citas", de modo que al recargar el navegador no se repita la operación.

Paso 1. Crear el proyecto y la estructura de carpetas

```
InmobiliariaWeb/ # raiz publica de la aplicacion
|-- index.jsp
|-- login.jsp
|-- acceso.jsp
|-- logout.jsp
|-- registro.jsp
|-- registrar.jsp
|-- landing.jsp
|-- inicio.jsp
|-- reportes.jsp

|-- css/
|   |-- estilos.css

|-- propiedad/
|   |-- listar.jsp
|   |-- nueva.jsp
|   |-- editar.jsp
|   |-- detalle.jsp
|   |-- acciones.jsp
|
|-- cita/
|   |-- nueva.jsp
|   |-- acciones.jsp
|   |-- mis-citas.jsp
|   |-- agenda.jsp
|
|-- solicitud/
|   |-- nueva.jsp
|   |-- acciones.jsp
|   |-- mis-solicitudes.jsp
|   |-- gestion.jsp
|
|-- admin/
|   |-- usuarios.jsp
|   |-- acciones.jsp
|
|-- pruebas/ # Java plano, NO se despliega en Tomcat
|   |-- Utilidades.java
|   |-- PruebasUnitarias.java
|
|-- scrum/
|   |-- product-backlog.md
|   |-- sprint1.md
|   |-- sprint2.md
|   |-- sprint3.md
|
|-- Evidencias/ # capturas de pantalla

-- WEB-INF/
|   |-- web.xml
|   |-- lib/
|   |   |-- mysql-connector-j-26.7.0.jar
|   |-- jspf/
|   |   |-- conexion.jspf
|   |   |-- utilidades.jspf
|   |   |-- seguridad.jspf
|   |   |-- sesionOpcional.jspf
|   |   |-- cabecera.jspf
|   |   |-- pie.jspf
```

Carpeta	Para qué sirve	¿Visible desde el navegador?
raíz	Páginas que abre el usuario directamente	Sí
Propiedad, cita, solicitud, admin	Un modulo por funcionalidad; facilita el control de acceso	Sí
css	Hoja de estilos propia	Sí
Pruebas	Código Java de las pruebas unitarias, aparte del proyecto web	No
Scrum, evidencias	Documentación del proyecto	No

WEB-INF	Configuración y fragmentos	No
WEB-INF/lib	Librerías .jar que usa la aplicación	No
WEB-INF/jspf	Fragmentos reutilizables	No

Paso 2. Instalar el driver JDBC de MySQL

Java no sabe hablar con MySQL por sí solo: necesita el driver, que es un archivo .jar. Descárguelo del sitio oficial de MySQL (Connector/J, versión 26.x o la que corresponda a su instalación, opción "Platform Independent") y copie el archivo mysql-connector-j-26.7.0.jar dentro de la carpeta WEB-INF/lib del proyecto.

Entorno	Dónde poner el .jar
VS Code (Estructura manual, como este proyecto)	Copiarlo directo a InmobiliariaWeb\WEB-INF\lib\.. No requiere configuración adicional en el editor.
Eclipse (Dynamic Web Project)	Arrastrarlo a WebContent/WEB-INF/lib; se agrega solo al classpath.
Maven	Agregar la dependencia com.mysql:mysql-connector-j en el pom.xml.

Importante: si el .jar no está dentro de WEB-INF/lib, el proyecto compila pero al ejecutarlo aparece el error `ClassNotFoundException: com.mysql.cj.jdbc.Driver`, o el mensaje personalizado que definimos en `conexion.jspf`: *"No se encontró el driver de MySQL."* Es uno de los errores más comunes del proyecto — te pasó justo así al principio y lo resolvimos copiando el .jar a esa carpeta y reiniciando Tomcat desde el panel de XAMPP.

3. Crear la base de datos y los usuarios de prueba

Abra MySQL Workbench (o la consola de MySQL) y ejecute el script `01_base_datos_inmobiliaria_completo.sql` que acompaña al proyecto. El script crea el esquema `inmobiliaria_simulacro`, las 13 tablas con sus llaves y restricciones, los catálogos (roles, ciudades, tipos de propiedad, características), y los datos de prueba (10 usuarios, 10 propiedades, 10 citas, 10 solicitudes, entre otros).

Paso 3.1 Usuarios con los que inicia sesión

Al final del script se insertan 10 usuarios. La clave de todos es 1234, pero en la tabla no se guarda '1234': Se guarda su resumen SHA-256 calculadora sobre el texto usuario:clave, exactamente igual que hace después el método claveCifrada() de utilidades.jspf

sql/01_base_datos_inmobiliaria_completo.sql (fragmento)

```
-- Password de todos los usuarios de prueba: 1234
INSERT INTO usuario (documento, username, correo, password_hash, id_rol, activo) VALUES
('1000000001', 'admin', 'admin@inmosim.com',
'f8e68e8d44bfb5314974a97f787d017ff6ac9d0046083f28665fcf96f0cef80c', 1, 1),
('1000000002', 'agente1', 'agente1@inmosim.com',
'6f819d06ab2da586d80b35e08cbfd67c04211babalaf350d709a73e981f43d96', 2, 1),
('1000000010', 'agente2', 'agente2@inmosim.com',
'c447b68b22a2588633531060f58c14elaab64c68b1fb29c583fda59559369d0', 2, 1),
('1000000011', 'agente3', 'agente3@inmosim.com',
'd0e94bfff017celfc9ab8875ef3123ffe2d206c5576b824eaad8d73b130433103', 2, 1),
('1000000012', 'agente4', 'agente4@inmosim.com',
'6245d8cef9a4fd73e69e0da520d65322bb906e012912c5f4049530a77355bda7', 2, 1),
('1000000003', 'cliente1', 'cliente1@inmosim.com',
'8929c861a6048eb04b8e12fcc39669bb4aceac129ae2b521b40ec035ed00244b', 3, 1),
('1000000020', 'cliente2', 'cliente2@inmosim.com',
'fb3e975a138b56806a1fc691a2ba318de0a233f84df72fc9e85384efalf4e8af', 3, 1),
('1000000021', 'cliente3', 'cliente3@inmosim.com',
'6a93a1f7b327cda19f69a4ad6c1e7c721e6ea8fae5fa493488b728b2cddb2e7', 3, 1),
('1000000022', 'cliente4', 'cliente4@inmosim.com',
'62538e91e6172c4958c116f8f809e95eefa08f2c7b833dc5296f93b8191318a6', 3, 1),
('1000000023', 'cliente5', 'cliente5@inmosim.com',
'90e8f7707425ffe22fbcd21eb6ab8a6dcb306fefel96ea292b69502b0d5f4e2', 3, 1);
```

Usuario	Clave	Rol	Qué puede hacer
Admin	1234	Administrador	Gestiona usuarios, roles y ve reportes
Agente1 – Agente4	1234	Inmobiliaria	Publican y administran sus propias propiedades
Cliente1 – Cliente5	1234	Cliente	Buscan propiedades, agendan citas y radican solicitudes

Paso 3.2 Verificación

Ejecute estas dos consultas: La primera debe devolver 13 y la segunda debe listar los 10 usuarios con su rol.

```
SELECT COUNT(*) AS tablas FROM information_schema.tables
WHERE table_schema = 'inmobiliaria_simulacro';

SELECT u.username, r.nombre AS rol FROM usuario u
JOIN rol r ON r.id_rol = u.id_rol ORDER BY u.id_usuario;
```

Paso 4. El archivo de conexión: conexion.jspf

Este es el archivo más importante del proyecto. Centraliza en un solo lugar los datos de acceso a MySQL y los métodos para abrir y cerrar la conexión. Si mañana el proyecto se lleva a otro computador, solo se cambian las cuatro constantes y toda la aplicación sigue funcionando.

WEB-INF/jspf/conexion.jspf

```
<%@ page import="java.sql.*" %>
<%!
    public static final String DB_DRIVER = "com.mysql.cj.jdbc.Driver";
    public static final String DB_URL =
        "jdbc:mysql://127.0.0.1:3306/inmobiliaria_simulacro"
        + "?useSSL=false&allowPublicKeyRetrieval=true"
        + "&serverTimezone=America/Bogota&characterEncoding=UTF-8";
    public static final String DB_USUARIO = "root";
    public static final String DB_CLAVE = "*****";

    public Connection abrirConexion() throws SQLException {
        try {
            Class.forName(DB_DRIVER);
        } catch (ClassNotFoundException ex) {
            throw new SQLException("No se encontro el driver de MySQL.", ex);
        }
        return DriverManager.getConnection(DB_URL, DB_USUARIO, DB_CLAVE);
    }

    public void cerrar(AutoCloseable... recursos) {
        for (int i = recursos.length - 1; i >= 0; i--) {
            if (recursos[i] != null) {
                try { recursos[i].close(); } catch (Exception ignorada) { }
            }
        }
    }

    public void deshacer(Connection con) {
        if (con != null) {
            try { con.rollback(); } catch (SQLException ignorada) { }
        }
    }
%>
```

Paso 4.1 Explicación parte por parte

Elemento	Qué hace y por qué está ahí
<%! ... %>	Bloque de declaración: lo que se escribe aquí se convierte en atributos y métodos de la clase del servlet, no en código del método de servicio. Por eso pueden declararse constantes y métodos.
DB_DRIVER	Nombre de la clase del driver. En el conector 26.x sigue siendo com.mysql.cj.jdbc.Driver.
DB_URL	Dirección de la base de datos con sus parámetros.
DB_USUARIO / DB_CLAVE	Credenciales de MySQL. Cámbielas por las de su equipo.
abrirConexion()	Carga el driver con Class.forName y devuelve una conexión nueva con DriverManager.getConnection.
cerrar(AutoCloseable...)	Recibe cualquier cantidad de recursos y los cierra en orden inverso, ignorando los errores del cierre. Evita repetir bloques finally enormes.
deshacer(Connection)	Ejecuta rollback() de forma segura cuando una transacción falla.

Paso 4.2 La URL de conexión, parámetro por parámetro

Parte de la URL	Significado
jdbc:mysql://	Protocolo JDBC y motor al que se conecta.
127.0.0.1:3306	Servidor y puerto de MySQL.
/inmobiliaria_simulacro	Nombre del esquema creado en el Paso 3.
useSSL=false	Evita la advertencia de conexión no cifrada en entornos locales.
allowPublicKeyRetrieval=true	Necesario con la autenticación por defecto de MySQL 8.
serverTimezone=America/Bogota	Sin este parámetro MySQL 8 puede lanzar un error de zona horaria.
characterEncoding=UTF-8	Permite guardar tildes y la letra ñ correctamente.

Paso 4.3 Probar la conexión antes de seguir

No avance sin comprobar que la conexión funciona. Ahí va el código de la página de prueba

prueba_conexion.jsp (pagina temporal, se borra al terminar)

```
<%@ page contentType="text/html" pageEncoding="UTF-8" %>
<%@ include file="/WEB-INF/jspf/conexion.jspf" %>
<html><body>
<%
    Connection con = null; Statement st = null; ResultSet rs = null;
    try {
        con = abrirConexion();
        st = con.createStatement();
        rs = st.executeQuery("SELECT COUNT(*) AS total FROM propiedad");
        rs.next();
%>
        <h4>Conexion exitosa</h4>
        Base de datos: <%= con.getCatalog() %><br>
        Propiedades registradas: <%= rs.getInt("total") %>
<%
    } catch (SQLException ex) {
%>
        <h4>Fallo la conexion</h4>
        <pre><%= ex.getMessage() %></pre>
<%
    } finally { cerrar(rs, st, con); }
%>
</body></html>
```

Abra http://localhost:8080/InmobiliariaWeb/prueba_conexion.jsp. Si aparece el aviso verde con el número de propiedades (10), la conexión quedó lista, y ya la probaste con éxito hace días.

Paso 5. Métodos de apoyo: Utilidades.jspf

El segundo fragmento reúne pequeñas funciones que se repetirían en todas las páginas: cifrar la contraseña, mostrar los precios en pesos, convertir los parámetros del formulario a número, y escapar el texto que escribe el usuario.

WEB-INF/jspf/utilidades.jspf

```
<%@ page import="java.security.MessageDigest, java.text.NumberFormat, java.util.Locale" %>
<%!
    public String sha256(String texto) {
        try {
            MessageDigest md = MessageDigest.getInstance("SHA-256");
            byte[] resumen = md.digest(texto.getBytes("UTF-8"));
            StringBuilder sb = new StringBuilder();
            for (byte b : resumen) sb.append(String.format("%02x", b));
            return sb.toString();
        } catch (Exception ex) {
            throw new RuntimeException("Error al cifrar la clave", ex);
        }
    }

    public String claveCifrada(String usuario, String clave) {
        return sha256(usuario + ":" + clave);
    }

    public String pesos(double valor) {
        NumberFormat f = NumberFormat.getInstance(new Locale("es", "CO"));
        f.setMaximumFractionDigits(0);
        return "$ " + f.format(valor);
    }

    public int aEntero(String valor, int porDefecto) {
        try { return Integer.parseInt(valor.trim()); }
        catch (Exception ex) { return porDefecto; }
    }

    public double aDoble(String valor, double porDefecto) {
        try { return Double.parseDouble(valor.trim().replace(",", ".")); }
        catch (Exception ex) { return porDefecto; }
    }

    public String esc(String texto) {
        if (texto == null) return "";
        return texto.replace("&", "&amp;").replace("<", "&lt;")
            .replace(">", "&gt;").replace("\\", "&quot;");
    }

    public String colorEstado(String estado) {
        if (estado == null) return "secondary";
        switch (estado) {
            case "DISPONIBLE": case "CONFIRMADA": case "APROBADA": return "success";
            case "RESERVADA": case "PENDIENTE": return "warning";
            case "VENDIDA": case "REALIZADA": return "primary";
            case "INACTIVA": case "CANCELADA": case "RECHAZADA": return "secondary";
            default: return "secondary";
        }
    }
}
%>
```

Paso 5.1 Para qué sirve cada método

Método	Uso en la aplicación
sha256(texto)	Calcula el resumen criptográfico. Es de una sola vía: del hash no se puede volver a la clave.
claveCifrada(usuario, clave)	Une el usuario y la clave antes de cifrar. Así dos usuarios con la misma clave tienen hashes distintos.
pesos(valor)	Convierte 320000000 en "\$ 320.000.000". Se usa en todas las tablas de precios.
aEntero(valor, defecto)	Convierte un parámetro de texto en entero sin que la página se caiga si viene vacío o alterado.
aDoble(valor, defecto)	Igual que el anterior para valores con decimales, como el precio de una propiedad.
esc(texto)	Reemplaza < > & " por sus entidades HTML. Impide que alguien escriba una etiqueta <script> en la descripción de una propiedad (ataque XSS).
colorEstado(estado)	Devuelve el color de Bootstrap del estado de una propiedad, cita o solicitud, para pintar las etiquetas de colores.

Paso 6. La plantilla visual con Bootstrap 5

Toda página del sistema tiene la misma estructura: encabezado con Bootstrap, barra de navegación que cambia según el rol, contenido, y pie. Se parte en dos fragmentos para no repetir ese HTML en cada archivo: cabecera.jspf y pie.jspf.

Paso 6.1 Cabecera.jspf

WEB-INF/jspf/cabecera.jspf

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title><%= tituloPagina %> | Inmobiliaria</title>
  <link href="https://cdn.jsdelivr.net/.../bootstrap.min.css" rel="stylesheet">
  <link href="<%= ctx %>/css/estilos.css" rel="stylesheet">
</head>
<body class="bg-light">
<nav class="navbar navbar-expand-lg navbar-dark bg-dark shadow-sm">
  <div class="container">
    <a class="navbar-brand fw-bold" href="<%= ctx %>/index.jsp">Inmobiliaria</a>
    <div class="collapse navbar-collapse" id="menu">
      <ul class="navbar-nav me-auto">
        <li><a href="<%= ctx %>/propiedad/listar.jsp">Propiedades</a></li>
        <% if ("INMOBILIARIA".equals(rolSesion) || "ADMINISTRADOR".equals(rolSesion)) { %>
          <li><a href="<%= ctx %>/propiedad/nueva.jsp">Nueva propiedad</a></li>
        <% } %>
        <% if ("CLIENTE".equals(rolSesion)) { %>
          <li><a href="<%= ctx %>/cita/mis-citas.jsp">Mis citas</a></li>
          <li><a href="<%= ctx %>/solicitud/mis-solicitudes.jsp">Mis solicitudes</a></li>
        <% } %>
        <% if ("INMOBILIARIA".equals(rolSesion) || "ADMINISTRADOR".equals(rolSesion)) { %>
          <li><a href="<%= ctx %>/cita/agenda.jsp">Agenda</a></li>
          <li><a href="<%= ctx %>/solicitud/gestion.jsp">Solicitudes</a></li>
        <% } %>
        <% if ("ADMINISTRADOR".equals(rolSesion)) { %>
          <li><a href="<%= ctx %>/reportes.jsp">Reportes</a></li>
          <li><a href="<%= ctx %>/admin/usuarios.jsp">Usuarios</a></li>
        <% } %>
      </ul>
      <% if (idUsuarioSesion != null) { %>
        <span class="navbar-text text-white me-3"><%= esc(nombreSesion) %>
        <span class="badge text-bg-warning"><%= rolSesion %></span></span>
        <a class="btn btn-outline-light btn-sm" href="<%= ctx %>/logout.jsp">Salir</a>
      <% } else { %>
        <a class="btn btn-outline-light btn-sm me-2" href="<%= ctx %>/login.jsp">Ingresar</a>
        <a class="btn btn-warning btn-sm fw-bold" href="<%= ctx %>/registro.jsp">Registrarme</a>
      <% } %>
    </div>
  </div>
</nav>
<main class="container my-4">
```

Paso 6.2 Qué aporta Bootstrap 5 aquí

Clase o componente	Efecto visual
navbar navbar-expand-lg navbar-dark bg-dark	Barra de navegación oscura que se colapsa en un botón cuando la pantalla es pequeña.
container	Centra el contenido y le da márgenes automáticos según el ancho de la pantalla.

collapse navbar-collapse	Menú desplegable en móvil.
badge text-bg-warning	Etiqueta de color; se usa para el rol y para el estado de propiedades/citas/solicitudes.
btn btn-outline-light / btn-warning	Botones de acción con estilos distintos según su importancia.

Paso 6.3 pie.jspf

WEB-INF/jspf/pie.jspf

```

</main>
<footer class="text-center text-muted py-3 border-top bg-white">
    <small>Inmobiliaria &middot; Programacion Java</small>
</footer>
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>

```

Importante: el archivo bootstrap.bundle.min.js debe cargarse al final del documento, no en la cabecera. Si se carga antes que el HTML, los menús desplegables y las alertas descartables no responden.

Paso 6.4 estilos.css

css/estilos.css

```
/* estilos.css - ajustes propios sobre Bootstrap 5 */
body { font-family: "Segoe UI", Roboto, Arial, sans-serif; }

/* Efecto hover en las tarjetas de propiedades del catalogo */
.card:hover { box-shadow: 0 .5rem 1rem rgba(0,0,0,.15); transition: box-shadow .2s ease-in-out; }

/* Impresion de una confirmacion de cita o solicitud: oculta menus y botones */
@media print {
  nav, footer, .no-imprimir { display: none !important; }
  .card { border: none !important; }
}
```

Paso 7. Control de acceso: seguridad.jspf

Sin este fragmento cualquiera podría escribir la dirección admin/usuarios.jsp y entrar sin iniciar sesión. La idea es sencilla: cada página declara qué roles pueden verla y el fragmento se encarga

de comprobarlo.

WEB-INF/jspf/seguridad.jspf

```
<% - -
seguridad.jspf - Control de acceso por sesion y por rol.
Cada pagina protegida declara ANTES del include el arreglo con los roles
autorizados y luego incluye este fragmento:
    <% String[] rolesPermitidos = {"CLIENTE", "ADMINISTRADOR"}; %>
    <%@ include file="/WEB-INF/jspf/seguridad.jspf" %>
- - %>
<%
    Integer idUsuarioSesion = (Integer) session.getAttribute("idUsuario");
    String rolSesion = (String) session.getAttribute("rol");
    String nombreSesion = (String) session.getAttribute("nombre");
    String ctx = request.getContextPath();

    // 1) Si no hay sesion activa, regresa al login
    if (idUsuarioSesion == null) {
        response.sendRedirect(ctx + "/login.jsp?error=sesion");
        return;
    }

    // 2) Si el rol del usuario no esta autorizado en esta pagina, lo bloquea
    boolean autorizado = false;
    for (String r : rolesPermitidos) {
        if (r.equals(rolSesion)) { autorizado = true; break; }
    }
    if (!autorizado) {
        response.sendRedirect(ctx + "/inicio.jsp?error=permiso");
        return;
    }
%>
```

Paso 7.1 Cómo se usa en cada página

```
<%@ include file="/WEB-INF/jspf/conexion.jspf" %>
<%@ include file="/WEB-INF/jspf/utilidades.jspf" %>
<% String[] rolesPermitidos = {"CLIENTE", "ADMINISTRADOR"}; %>
<%@ include file="/WEB-INF/jspf/seguridad.jspf" %>
```

El orden importa. La variable rolesPermitidos se declara **antes** del include porque, al ser una inclusión estática, el fragmento se pega justo debajo y necesita que la variable ya exista.

Línea del fragmento	Explicación
---------------------	-------------

<code>session.getAttribute("idUsuario")</code>	Recupera el identificador guardado al iniciar sesión. Si es null, no hay sesión activa.
<code>request.getContextPath()</code>	Devuelve el nombre de la aplicación (/InmobiliariaWeb). Se guarda en ctx y se usa en todos los enlaces.
<code>response.sendRedirect(...)</code>	Envía al navegador a otra dirección.
<code>return;</code>	Corta la ejecución de la página. Sin este return, el resto del HTML se seguiría generando después de la redirección y saldría el error "response already committed".
<code>for (String r : rolesPermitidos)</code>	Recorre los roles autorizados y compara con el de la sesión.

Importante: los roles se escriben exactamente como están en la tabla rol: ADMINISTRADOR, INMOBILIARIA, CLIENTE. Una diferencia de mayúsculas hace que el usuario quede bloqueado en su propio módulo.

Existe también una variante de este fragmento, `sesionOpcional.jspf`, para páginas públicas (landing, catálogo) donde tanto un visitante como un usuario logueado pueden entrar — la diferencia es que **no redirige** si no hay sesión, solo asigna `rolSesion = "VISITANTE"` por defecto. Ya la construimos junto con `seguridad.jspf`, así que si quieres el código de esa también, dime y te lo paso.

Paso 8. Inicio y cierre de sesión

El acceso se resuelve con tres páginas: el formulario (`login.jsp`), el validador (`acceso.jsp`) y la salida (`logout.jsp`). Ninguna de las tres incluye `seguridad.jspf`, porque son justamente las páginas públicas.

Paso 8.1 login.jsp

login.jsp

```
<%--
login.jsp - Formulario de inicio de sesion. Es la unica pagina publica
de autenticacion: no incluye seguridad.jspf.
--%>
<%@ page contentType="text/html" pageEncoding="UTF-8" %>
<%
    String ctx = request.getContextPath();
    String error = request.getParameter("error");
    String mensaje = null;
    if ("clave".equals(error))    mensaje = "Usuario o clave incorrectos.";
    if ("sesion".equals(error))  mensaje = "Debe iniciar sesion para continuar.";
    if ("vacio".equals(error))   mensaje = "Escriba el usuario y la clave.";
    if ("inactivo".equals(error)) mensaje = "El usuario esta inhabilitado.";
%>
<!DOCTYPE html>
<html lang="es">
<body class="bg-dark d-flex align-items-center" style="min-height:100vh">
<div class="container" style="max-width:420px">
    <div class="card shadow-lg border-0">
        <div class="card-body p-4">
            <% if (mensaje != null) { %>
                <div class="alert alert-danger py-2"><%= mensaje %></div>
            <% } %>
            <form method="post" action="<%= ctx %>/acceso.jsp">
                <input type="text" name="usuario" required autofocus placeholder="admin">
                <input type="password" name="clave" required placeholder="*****">
                <button type="submit">Ingresar</button>
            </form>
            <p>¿No tienes cuenta? <a href="<%= ctx %>/registro.jsp">Registrate</a></p>
        </div>
    </div>
</div>
</body>
</html>
```

Paso 8.2 acceso.jsp

acceso.jsp

```
<%--
acceso.jsp - Valida las credenciales contra la tabla usuario.
No genera HTML: solo procesa y redirige.
--%>
<%@ page contentType="text/html" pageEncoding="UTF-8" %>
<%@ include file="/WEB-INF/jspf/conexion.jspf" %>
<%@ include file="/WEB-INF/jspf/utilidades.jspf" %>
<%
    request.setCharacterEncoding("UTF-8");
    String ctx = request.getContextPath();
    String usuario = request.getParameter("usuario");
    String clave = request.getParameter("clave");

    if (usuario == null || clave == null
        || usuario.trim().isEmpty() || clave.trim().isEmpty()) {
        response.sendRedirect(ctx + "/login.jsp?error=vacio");
        return;
    }
    usuario = usuario.trim();

    Connection con = null; PreparedStatement ps = null; ResultSet rs = null;
    try {
        con = abrirConexion();
        String sql = "SELECT u.id_usuario, u.password_hash, u.activo, "
            + "        p.nombres, p.apellidos, r.nombre AS rol "
            + "FROM usuario u "
            + " JOIN rol r ON r.id_rol = u.id_rol "
            + " LEFT JOIN perfil p ON p.id_usuario = u.id_usuario "
            + "WHERE u.username = ?";
        ps = con.prepareStatement(sql);
        ps.setString(1, usuario);
        rs = ps.executeQuery();

        if (rs.next()) {
            String hashGuardado = rs.getString("password_hash");
            boolean activo = rs.getBoolean("activo");
            if (!activo) {
                response.sendRedirect(ctx + "/login.jsp?error=inactive");
                return;
            }
            if (hashGuardado.equals(claveCifrada(usuario, clave))) {
                session.setAttribute("idUser", rs.getInt("id_usuario"));
                session.setAttribute("nombre",
                    rs.getString("nombres") + " " + rs.getString("apellidos"));
                session.setAttribute("rol", rs.getString("rol"));
                session.setAttribute("username", usuario);
                session.setMaxInactiveInterval(30 * 60);
                response.sendRedirect(ctx + "/inicio.jsp");
                return;
            }
        }
        response.sendRedirect(ctx + "/login.jsp?error=clave");
    } catch (SQLException ex) {
        out.println("Error de conexion con la base de datos: " + ex.getMessage());
    } finally {
        cerrar(rs, ps, con);
    }
%>
```

Paso 8.3 Los cuatro puntos clave de la validación

Punto	Explicación
PreparedStatement con ?	La consulta se envía con un hueco y el valor viaja aparte. Si alguien escribe ' OR '1'='1 como usuario, se busca literalmente ese texto y no se altera la consulta.
Comparación de hashes	Nunca se consulta WHERE clave = ?. Se trae el hash guardado y se compara con el hash de lo que escribió el usuario.
Atributos de sesión	Se guardan idUsuario, nombre, rol y username. La clave jamás se guarda en la sesión.
setMaxInactiveInterval(30*60)	La sesión caduca a los 30 minutos sin actividad.

Paso 8.4 logout.jsp

```
<%-- logout.jsp - Cierra la sesion y regresa al login. --%>
<%@ page contentType="text/html" pageEncoding="UTF-8" %>
<%
    session.invalidate();
    response.sendRedirect("login.jsp");
%>
```

invalidate() destruye la sesión completa: todos los atributos desaparecen y el siguiente acceso a cualquier página protegida vuelve al login.

Paso 8.5 Registro público: registro.jsp y registrar.jsp

registrar.jsp - Controlador del registro publico

```
<%-
registrar.jsp - Inserta usuario + perfil en una transaccion. Rol CLIENTE (id 3) por defecto.
-%>
<%@ page contentType="text/html" pageEncoding="UTF-8" %>
<%@ include file="/WEB-INF/jspf/conexion.jspf" %>
<%@ include file="/WEB-INF/jspf/utilidades.jspf" %>
<%
    request.setCharacterEncoding("UTF-8");
    String ctx = request.getContextPath();
    String nombres = request.getParameter("nombres");
    String apellidos = request.getParameter("apellidos");
    String documento = request.getParameter("documento");
    String telefono = request.getParameter("telefono");
    String direccion = request.getParameter("direccion");
    String username = request.getParameter("username");
    String correo = request.getParameter("correo");
    String clave = request.getParameter("clave");
    String clave2 = request.getParameter("clave2");

    if (!clave.equals(clave2)) {
        response.sendRedirect(ctx + "/registro.jsp?err="
            + java.net.URLEncoder.encode("Las contraseñas no coinciden.", "UTF-8"));
        return;
    }

    Connection con = null; PreparedStatement ps = null; ResultSet rs = null;
    try {
        con = abrirConexion();
        con.setAutoCommit(false);

        ps = con.prepareStatement(
            "INSERT INTO usuario (documento, username, correo, password_hash, id_rol, activo)
            + "VALUES (?, ?, ?, ?, 3, 1)", Statement.RETURN_GENERATED_KEYS);
        ps.setString(1, documento.trim());
        ps.setString(2, username.trim());
        ps.setString(3, correo.trim());
        ps.setString(4, claveCifrada(username.trim(), clave));
        ps.executeUpdate();
        rs = ps.getGeneratedKeys();
        rs.next();
        int idUsuario = rs.getInt(1);
        cerrar(rs, ps);

        ps = con.prepareStatement(
            "INSERT INTO perfil (id_usuario, nombres, apellidos, telefono, direccion) "
            + "VALUES (?, ?, ?, ?, ?)");
        ps.setInt(1, idUsuario);
        ps.setString(2, nombres.trim());
        ps.setString(3, apellidos.trim());
        ps.setString(4, telefono);
        ps.setString(5, direccion);
        ps.executeUpdate();
        cerrar(ps);

        con.commit();
        response.sendRedirect(ctx + "/login.jsp?msg=cuenta_creada");

    } catch (SQLException ex) {
        deshacer(con);
        String m = ex.getMessage();
        String amigable = m.contains("uq_usuario_correo") ? "Ese correo ya está registrado."
            : m.contains("uq_usuario_username") ? "Ese nombre de usuario ya existe."
            : m.contains("uq_usuario_documento") ? "Ese documento ya está registrado."
            : "Datos duplicados.";
        response.sendRedirect(ctx + "/registro.jsp?err=" + java.net.URLEncoder.encode(amigable
            , "UTF-8"));
    } catch (SQLException ex) {
        deshacer(con);
        response.sendRedirect(ctx + "/registro.jsp?err=" + java.net.URLEncoder.encode(ex.getMe
            ssage(), "UTF-8"));
    } finally {
        cerrar(rs, ps, con);
    }
}%>
```

Puntos clave	Explicación
<code>con.setAutoCommit(false)</code>	Inicia una transacción: usuario y perfil se guardan juntos o ninguno de los dos queda aplicado.
<code>id_rol = 3</code> (fijo en el SQL)	Todo registro público entra como CLIENTE por defecto; solo el administrador puede cambiar el rol después, desde <code>admin/usuarios.jsp</code> .
<code>Statement.RETURN_GENERATED_KEYS</code>	Recupera el <code>id_usuario</code> recién insertado, necesario para el segundo INSERT (perfil).
<code>SQLIntegrityConstraintViolationException</code>	Se captura aparte para traducir la violación de UNIQUE (correo/username/documento duplicado) a un mensaje claro, distinguiendo cuál de los tres campos fue.

El formulario `registro.jsp` en sí es HTML+Bootstrap estándar (campos de texto con `required`, un `type="email"` para el correo, y `minlength="4"` en las contraseñas)

Paso 8.6 `index.jsp`

```
<%-- index.jsp - Redirige al panel si hay sesion, o a la landing si no. --%>
<%@ page contentType="text/html" pageEncoding="UTF-8" %>
<%
    if (session.getAttribute("idUsuario") != null) {
        response.sendRedirect("inicio.jsp");
    } else {
        response.sendRedirect("landing.jsp");
    }
%>
```

Es la puerta de entrada de toda la aplicación (`http://localhost:8080/InmobiliariaWeb/`). Nota que este archivo **cambió respecto al del Simulacro**: antes redirigía siempre a `login.jsp` si no había sesión; ahora redirige a `landing.jsp`, porque el enunciado exige que el visitante pueda navegar el catálogo público sin que se le obligue a iniciar sesión primero.

Paso 8.7 landing.jsp

landing.jsp

```
<%@ page contentType="text/html" pageEncoding="UTF-8" %>
<%@ include file="/WEB-INF/jspf/conexion.jspf" %>
<%@ include file="/WEB-INF/jspf/utilidades.jspf" %>
<%@ include file="/WEB-INF/jspf/sesionOpcional.jspf" %>
<% String tituloPagina = "Inicio"; %>
<%@ include file="/WEB-INF/jspf/cabecera.jspf" %>

<div class="p-5 mb-4 bg-dark rounded-3 text-white">
  <h1>Encuentra tu proximo hogar</h1>
  <form method="get" action="<%= ctx %>/propiedad/listar.jsp">
    <input type="text" name="q" placeholder="Buscar por titulo">
    <select name="ciudad">
      <!-- opciones desde SELECT id_ciudad, nombre FROM ciudad --%>
    </select>
    <button>Buscar</button>
  </form>
</div>

<h4>Propiedades destacadas</h4>
<%
  rs = st.executeQuery(
    "SELECT pr.id_propiedad, pr.titulo, pr.precio, c.nombre AS ciudad, "
    + " (SELECT url_imagen FROM imagen_propiedad ip WHERE ip.id_propiedad = pr
.id_propiedad "
    + " ORDER BY orden LIMIT 1) AS imagen "
    + "FROM propiedad pr JOIN ciudad c ON c.id_ciudad = pr.id_ciudad "
    + "WHERE pr.estado = 'DISPONIBLE' ORDER BY pr.fecha_publicacion DESC LIMIT
3");
  // ... una tarjeta Bootstrap con imagen, ciudad y precio por cada propiedad d
estacada ...
%>

<%@ include file="/WEB-INF/jspf/pie.jspf" %>
```

Importante: sesionOpcional.jspf (no seguridad.jspf) es lo que permite que esta sea la primera página que ve un visitante sin sesión. Y la subconsulta de imagen (SELECT url_imagen ... LIMIT 1) es el mismo patrón que se repite en listar.jsp y detalle.jsp — vale la pena mencionarlo en la sustentación como ejemplo de reutilizar un mismo patrón SQL en varios lugares.

Paso 9. El panel principal: inicio.jsp

Es la primera página que se ve tras iniciar sesión. Muestra un indicador del día y las tarjetas de acceso a los módulos según el rol del usuario.

inicio.jsp

```
<%--
    inicio.jsp - Panel principal. Muestra un indicador del dia y los accesos
    directos que corresponden al rol del usuario.
--%>
<%@ page contentType="text/html" pageEncoding="UTF-8" %>
<%@ include file="/WEB-INF/jspf/conexion.jspf" %>
<%@ include file="/WEB-INF/jspf/utilidades.jspf" %>
<% String[] rolesPermitidos = {"ADMINISTRADOR", "INMOBILIARIA", "CLIENTE"}; %>
<%@ include file="/WEB-INF/jspf/seguridad.jspf" %>
<%
    String tituloPagina = "Inicio";
    int totalPropiedades = 0;
    Connection con = null; Statement st = null; ResultSet rs = null;
    try {
        con = abrirConexion();
        st = con.createStatement();
        rs = st.executeQuery("SELECT COUNT(*) AS total FROM propiedad WHERE estado = '
DISPONIBLE'");
        if (rs.next()) totalPropiedades = rs.getInt("total");
    } catch (SQLException ex) {
        request.setAttribute("errorBD", ex.getMessage());
    } finally { cerrar(rs, st, con); }
%>
<%@ include file="/WEB-INF/jspf/cabecera.jspf" %>

<h3 class="mb-1">Hola, <%= esc(nombreSesion) %></h3>
<p class="text-muted">Panel de control del rol <%= rolSesion.toLowerCase() %>.</p>

<% if ("permiso".equals(request.getParameter("error"))) { %>
    <div class="alert alert-warning">No tiene permisos para entrar a esa seccion.</div>
>
<% } %>

<div class="row g-3 mb-4">
    <div class="col-6 col-lg-3">
        <div class="card border-0 shadow-sm text-center h-100">
            <div class="card-body">
                <h3 class="mt-2 mb-0"><%= totalPropiedades %></h3>
                <small class="text-muted">Propiedades disponibles</small>
            </div>
        </div>
    </div>
</div>

<div class="row g-3">
    <% if ("INMOBILIARIA".equals(rolSesion) || "ADMINISTRADOR".equals(rolSesion)) { %>
        <div class="col-md-4"><div class="card h-100 shadow-sm"><div class="card-body">
            <h5>Gestionar propiedades</h5>
            <a href="<%= ctx %>/propiedad/listar.jsp" class="btn btn-warning btn-sm">E
ntrar</a>
        </div></div></div>
        <% } %>
        <% if ("CLIENTE".equals(rolSesion)) { %>
            <div class="col-md-4"><div class="card h-100 shadow-sm"><div class="card-body">
                <h5>Buscar propiedades</h5>
                <a href="<%= ctx %>/propiedad/listar.jsp" class="btn btn-primary btn-sm">E
ntrar</a>
            </div></div></div>
            <% } %>
        </div>
    </div>

<%@ include file="/WEB-INF/jspf/pie.jspf" %>
```

Paso 9.1 Detalles que vale la pena mirar

Detalles que vale la pena mirar	Explicación
rolesPermitidos = los 3 roles	Cualquier usuario logueado (no visitante) puede ver su panel — VISITANTE no aparece porque esta página exige sesión.
SELECT COUNT(*) ... WHERE estado='DISPONIBLE'	Un indicador simple pero real, tomado directo de la base de datos, no un número inventado.
Tarjetas condicionales por rol	Cada bloque if muestra solo las opciones relevantes: el agente ve "Gestionar", el cliente ve "Buscar" — ambos enlazan al mismo listar.jsp, pero ese archivo decide internamente qué botones mostrarle a cada uno.
request.getParameter("error").equals("permiso")	Si seguridad.jspf te rebotó desde otra página por no tener el rol correcto, aquí es donde aterrizas con el aviso amarillo.

Paso 10. Módulo de propiedades

Es el módulo más extenso porque concentra el corazón del negocio: catálogo, publicación, ficha de detalle, edición y baja.

Paso 10.1 listar.jsp: El catálogo con filtros

propiedad/listar.jsp

```
<%@ include file="/WEB-INF/jspf/sesionOpcional.jspf" %>
<%
    String q = request.getParameter("q");
    String ciudadParam = request.getParameter("ciudad");
    String tipoParam = request.getParameter("tipo");
    String precioMin = request.getParameter("precioMin");
    String precioMax = request.getParameter("precioMax");
%>
<%-- ... filtros HTML (texto, selects de ciudad/tipo, precio min/max) ... --%>
<%
    StringBuilder sql = new StringBuilder(
        "SELECT pr.id_propiedad, pr.matricula_inmobiliaria, pr.titulo, pr.precio, pr.estado, "
        + "      c.nombre AS ciudad, t.nombre AS tipo, u.username AS agente, "
        + "      (SELECT url_imagen FROM imagen_propiedad ip WHERE ip.id_propiedad = pr.id_propiedad "
        + "      ORDER BY orden LIMIT 1) AS imagen "
        + "FROM propiedad pr "
        + " JOIN ciudad c ON c.id_ciudad = pr.id_ciudad "
        + " JOIN tipo_propiedad t ON t.id_tipo_propiedad = pr.id_tipo_propiedad "
        + " JOIN usuario u ON u.id_usuario = pr.id_usuario "
        + "WHERE pr.estado <> 'INACTIVA' ");
    if (q != null && !q.trim().isEmpty()) sql.append(" AND pr.titulo LIKE ? ");
    if (ciudadParam != null && !ciudadParam.trim().isEmpty()) sql.append(" AND pr.id_ciudad = ? ");
    if (tipoParam != null && !tipoParam.trim().isEmpty()) sql.append(" AND pr.id_tipo_propiedad = ? ");
    if (precioMin != null && !precioMin.trim().isEmpty()) sql.append(" AND pr.precio >= ? ");
    if (precioMax != null && !precioMax.trim().isEmpty()) sql.append(" AND pr.precio <= ? ");
    sql.append(" ORDER BY pr.fecha_publicacion DESC");

    ps = con.prepareStatement(sql.toString());
    int i = 1;
    if (q != null && !q.trim().isEmpty()) ps.setString(i++, "%" + q.trim() + "%");
    if (ciudadParam != null && !ciudadParam.trim().isEmpty()) ps.setInt(i++, aEntero(ciudadParam, 0));
    if (tipoParam != null && !tipoParam.trim().isEmpty()) ps.setInt(i++, aEntero(tipoParam, 0));
    if (precioMin != null && !precioMin.trim().isEmpty()) ps.setDouble(i++, aDoble(precioMin, 0));
    if (precioMax != null && !precioMax.trim().isEmpty()) ps.setDouble(i++, aDoble(precioMax, 0));
    rs = ps.executeQuery();
    // ... recorre rs y pinta una tarjeta Bootstrap por propiedad ...
%>
```

Detalle	Explicación
sesionOpcional.jspf	Esta página es pública: visitantes y usuarios logueados ven el catálogo con filtros.
StringBuilder sql + if por filtro	La consulta se arma dinámicamente: solo agrega el pedazo AND de cada filtro que el usuario realmente envió.
int i = 1; con i++ repetido	El índice del PreparedStatement debe coincidir exactamente con el orden en que se agregaron los ? al StringBuilder — por eso los mismos if se repiten dos veces, una para construir el texto y otra para asignar los valores.
LIKE ? con "%" + q.trim() + "%"	Búsqueda parcial: encuentra el texto en cualquier parte del título, no solo al inicio.
Subconsulta de imagen	Trae solo la primera imagen de la galería (ORDER BY orden LIMIT 1) para la miniatura de cada tarjeta.

Importante: Esta consulta es, además, la **consulta obligatoria #1** de tu parcial (INNER JOIN de 4 tablas: propiedad + ciudad + tipo_propiedad + usuario) — vale la pena que lo menciones explícitamente aquí y no solo en la sección de consultas.

Paso 10.2 nueva.jsp: Publicar una propiedad

propiedad/nueva.jsp

```
<%--
    propiedad/nueva.jsp - Formulario para publicar una propiedad nueva.
--%>
<%@ page contentType="text/html" pageEncoding="UTF-8" %>
<%@ include file="/WEB-INF/jspf/conexion.jspf" %>
<%@ include file="/WEB-INF/jspf/utilidades.jspf" %>
<% String[] rolesPermitidos = {"ADMINISTRADOR", "INMOBILIARIA"}; %>
<%@ include file="/WEB-INF/jspf/seguridad.jspf" %>
<% String tituloPagina = "Nueva propiedad"; %>
<%@ include file="/WEB-INF/jspf/cabecera.jspf" %>

<% if (request.getParameter("err") != null) { %>
    <div class="alert alert-danger"><%= esc(request.getParameter("err")) %></div>
<% } %>

<form method="post" action="<%= ctx %>/propiedad/acciones.jsp">
    <input type="hidden" name="accion" value="crear">
    <input type="text" name="matricula" required maxlength="30" placeholder="MI-001
1">
    <input type="text" name="titulo" required maxlength="120">
    <textarea name="descripcion" maxlength="500"></textarea>
    <input type="number" name="precio" required min="0" step="1000">

    <select name="id_ciudad" required>
        <!-- opciones generadas desde SELECT id_ciudad, nombre FROM ciudad --%>
    </select>
    <select name="id_tipo_propiedad" required>
        <!-- opciones generadas desde SELECT id_tipo_propiedad, nombre FROM tipo_pr
opiedad --%>
    </select>

    <!-- checkboxes de características, generados desde SELECT * FROM caracteristic
a --%>
    <input type="checkbox" name="caracteristicas" value="<id>">

    <button type="submit">Publicar</button>
</form>
```

Detalle	Explicación
rolesPermitidos = {"ADMINISTRADOR", "INMOBILIARIA"}	Solo estos dos roles pueden publicar; un CLIENTE que intente entrar aquí escribiendo la URL es bloqueado por seguridad.jspf.

Checkboxes con name="caracteristicas" repetido	Al tener el mismo name, todos los que el usuario marque llegan al controlador como un arreglo (String[]), no como valores sueltos.
Selects generados desde SELECT ... FROM ciudad / tipo_propiedad	Las opciones nunca están escritas a mano en el HTML: siempre vienen de la base de datos, así que si el admin agrega una ciudad nueva, aparece aquí automáticamente.

Paso 10.3 detalle.jsp: La ficha de la propiedad

Es la página más larga del proyecto. Se explica en cuatro bloques.

Bloque 1 de 4: Cargar los datos de la propiedad

propiedad/detalle.jsp (bloque 1 de 4)

```
<%@ page contentType="text/html" pageEncoding="UTF-8" %>
<%@ include file="/WEB-INF/jspf/conexion.jspf" %>
<%@ include file="/WEB-INF/jspf/utilidades.jspf" %>
<%@ include file="/WEB-INF/jspf/sesionOpcional.jspf" %>
<%
    String tituloPagina = "Detalle de propiedad";
    int idPropiedad = aEntero(request.getParameter("id"), 0);
    String msg = request.getParameter("msg");

    Connection con = null; PreparedStatement ps = null; ResultSet rs = null;
    boolean existe = false;
    String titulo = "", descripcion = "", ciudad = "", tipo = "", estado = "", matricula = "";
    double precio = 0;
    int idAgente = 0;
    try {
        con = abrirConexion();
        ps = con.prepareStatement(
            "SELECT pr.titulo, pr.descripcion, pr.precio, pr.estado, pr.matricula_inmobiliaria, "
            + "      c.nombre AS ciudad, t.nombre AS tipo, pr.id_usuario "
            + "FROM propiedad pr "
            + " JOIN ciudad c ON c.id_ciudad = pr.id_ciudad "
            + " JOIN tipo_propiedad t ON t.id_tipo_propiedad = pr.id_tipo_propiedad "
            + "WHERE pr.id_propiedad = ?");
        ps.setInt(1, idPropiedad);
        rs = ps.executeQuery();
        if (rs.next()) {
            existe = true;
            titulo = rs.getString("titulo");
            descripcion = rs.getString("descripcion");
            precio = rs.getDouble("precio");
            estado = rs.getString("estado");
            matricula = rs.getString("matricula_inmobiliaria");
            ciudad = rs.getString("ciudad");
            tipo = rs.getString("tipo");
            idAgente = rs.getInt("id_usuario");
        }
        cerrar(rs, ps);
    } catch (SQLException ex) {
        request.setAttribute("errorBD", ex.getMessage());
    }
%>
<%@ include file="/WEB-INF/jspf/cabecera.jspf" %>
```

Antes de imprimir nada se consulta la propiedad y se guarda su información en variables. Fíjate que `sesionOpcional.jspf` (no `seguridad.jspf`) es lo que permite que un visitante vea esta ficha sin sesión.

Bloque 2 de 4: La galería de imágenes

propiedad/detalle.jsp (bloque 2 de 4)

```
<% if (!existe) { %>
    <div class="alert alert-danger">La propiedad solicitada no existe.</div>
<% } else { %>

<div class="row g-4">
    <div class="col-lg-7">
        <!-- ===== Galeria (carrusel Bootstrap) ===== -->
        <div id="galeria" class="carousel slide" data-bs-ride="carousel">
            <div class="carousel-inner rounded">
<%
        try {
            ps = con.prepareStatement(
                "SELECT url_imagen FROM imagen_propiedad WHERE id_propiedad = ? ORDER
                BY orden");
            ps.setInt(1, idPropiedad);
            rs = ps.executeQuery();
            int i = 0;
            boolean alguna = false;
            while (rs.next()) {
                alguna = true;
<%
                <div class="carousel-item <%= i == 0 ? "active" : "" %>">
                    " class="d-block w-100"
                        style="height:380px;object-fit:cover">
                </div>
<%
                i++;
            }
            if (!alguna) {
<%
                <div class="carousel-item active">
                    
                </div>
<%
            }
            cerrar(rs, ps);
        } catch (SQLException ex) { }
<%
    </div>
    <button class="carousel-control-prev" data-bs-target="#galeria" data-bs-sli
de="prev"></button>
    <button class="carousel-control-next" data-bs-target="#galeria" data-bs-sli
de="next"></button>
</div>

<h3 class="mt-3"><%= esc(titulo) %></h3>
<p class="text-muted"><%= esc(ciudad) %> &middot; <%= esc(tipo) %>
    &middot; <code><%= esc(matricula) %></code>
    &middot; <span class="badge text-bg-<%= colorEstado(estado) %>"><%= estad
o %></span></p>
<p><%= esc(descripcion) %></p>
```

Si la propiedad no tiene ninguna imagen cargada, se muestra una de relleno (picsum.photos) en vez de dejar el carrusel vacío o roto.

Bloque 3 de 4: Características (Resuelve la relación N:M)

propiedad/detalle.jsp (bloque 3 de 4)

```
<h6>Características</h6>
<div class="mb-3">
<%
    try {
        ps = con.prepareStatement(
            "SELECT c.nombre FROM propiedad_caracteristica pc "
            + "JOIN caracteristica c ON c.id_caracteristica = pc.id_caract
eristica "
            + "WHERE pc.id_propiedad = ?");
        ps.setInt(1, idPropiedad);
        rs = ps.executeQuery();
        while (rs.next()) {
%>
            <span class="badge text-bg-light border me-1"><%= esc(rs.getStri
ng("nombre")) %></span>
<%
        }
        cerrar(rs, ps);
    } catch (SQLException ex) { }
%>
    </div>
</div>
```

Importante: esta consulta es la **consulta obligatoria #3** del parcial (la que resuelve la relación N:M). A diferencia de la del listado (que junta 4 tablas para mostrar el catálogo completo), esta trae las características de **una sola propiedad puntual** — es la diferencia que probablemente te pregunten si mencionas ambas consultas en la sustentación.

Bloque 4 de 4: El panel de acciones (Cambia según el rol)

propiedad/detalle.jsp (bloque 4 de 4)

```
<!-- ===== Panel de acciones ===== -->
<div class="col-lg-5">
  <div class="card shadow-sm">
    <div class="card-body">
      <h3 class="text-warning fw-bold"><%= pesos(precio) %></h3>

      <% if ("ADMINISTRADOR".equals(rolSesion)
          || (idUserioSesion != null && idUsuarioSesion == idAgente)) { %>
        <a href="<%= ctx %>/propiedad/editar.jsp?id=<%= idPropiedad %>"
          class="btn btn-outline-dark w-100 mb-2">Editar propiedad</a>
      <% } %>

      <% if ("CLIENTE".equals(rolSesion)) { %>
        <form method="post" action="<%= ctx %>/propiedad/acciones.jsp" class="d
-grid mb-2">
          <input type="hidden" name="accion" value="favorito">
          <input type="hidden" name="id_propiedad" value="<%= idPropiedad %>"
        >
          <button class="btn btn-outline-danger">Marcar/Quitar de favoritos</
button>
        </form>
        <a href="<%= ctx %>/cita/nueva.jsp?id=<%= idPropiedad %>"
          class="btn btn-primary w-100 mb-2">Agendar cita</a>
        <a href="<%= ctx %>/solicitud/nueva.jsp?id=<%= idPropiedad %>"
          class="btn btn-success w-100">Solicitar compra/arriendo</a>

      <% } else if (idUserioSesion == null) { %>
        <div class="alert alert-warning small">
          <a href="<%= ctx %>/login.jsp">Inicia sesion</a> o
          <a href="<%= ctx %>/registro.jsp">registrate</a> para agendar
          una cita o hacer una solicitud.
        </div>
      <% } %>
    </div>
  </div>
</div>
<% } %>
<%@ include file="/WEB-INF/jspf/pie.jspf" %>
```

Importante: el botón "Editar propiedad" solo aparece si `idUserioSesion == idAgente` (o eres administrador) — es decir, aunque cualquiera pueda **ver** la ficha (incluso un visitante), solo el dueño real de la propiedad ve el botón para modificarla. Esta comparación ocurre en el servidor, no es solo un if cosmético en el HTML.

Paso 10.4 editar.jsp: Edición de una propiedad

Reutiliza casi por completo la estructura de nueva.jsp (mismo formulario: título, descripción, precio, ciudad, tipo, características), con dos diferencias clave: primero consulta los datos actuales de la propiedad para precargar el formulario (en vez de empezar en blanco), y al enviar el formulario manda accion=editar en vez de accion=crear al controlador propiedad/acciones.jsp. También consulta qué características ya tenía marcadas la propiedad, para que esos checkboxes aparezcan ya seleccionados en vez de que el usuario tenga que volver a marcarlos todos desde cero.

El fragmento que vale la pena destacar es justo esa precarga:

propiedad/editar.jsp (fragmento: precarga de datos)

```
<%
    int idPropiedad = aEntero(request.getParameter("id"), 0);
    Set<Integer> caracSeleccionadas = new HashSet<>();

    ps = con.prepareStatement(
        "SELECT matricula_inmobiliaria, titulo, descripcion, precio, id_ciudad, id_tipo_propiedad "
        + "FROM propiedad WHERE id_propiedad = ?");
    ps.setInt(1, idPropiedad);
    rs = ps.executeQuery();
    if (rs.next()) {
        titulo = rs.getString("titulo");
        descripcion = rs.getString("descripcion");
        precio = rs.getDouble("precio");
        idCiudadActual = rs.getInt("id_ciudad");
        idTipoActual = rs.getInt("id_tipo_propiedad");
    }
    cerrar(rs, ps);

    // Que características ya tenía marcadas, para pre-seleccionar los checkboxes
    ps = con.prepareStatement(
        "SELECT id_caracteristica FROM propiedad_caracteristica WHERE id_propiedad = ?");
    ps.setInt(1, idPropiedad);
    rs = ps.executeQuery();
    while (rs.next()) caracSeleccionadas.add(rs.getInt("id_caracteristica"));
%>

<!-- Mas abajo, en cada checkbox del formulario: --%>
<input type="checkbox" name="caracteristicas" value="<%= idc %>"
    <%= caracSeleccionadas.contains(idc) ? "checked" : "" %>>
```

La idea clave para tu texto: `Set<Integer>.contains()` es lo que permite decirle a cada checkbox, uno por uno mientras se genera el HTML, si debe nacer ya marcado o no — sin eso, editar una propiedad "olvidaría" visualmente sus características aunque siguieran guardadas en la base de datos.

Paso 10.5 acciones.jsp: El controlador de propiedades

Aquí ocurre todo lo que modifica la base de datos. Es el mejor ejemplo de controlador con transacciones de todo el proyecto — lo dividimos en partes.

Estructura general (Preparación y doble validación de rol)

propiedad/acciones.jsp (parte 1: preparacion)

```
<!--
propiedad/acciones.jsp - Controlador del modulo de propiedades.
Acciones: crear | editar | baja | favorito
--%>
<%@ page contentType="text/html" pageEncoding="UTF-8" %>
<%@ include file="/WEB-INF/jspf/conexion.jspf" %>
<%@ include file="/WEB-INF/jspf/utilidades.jspf" %>
<% String[] rolesPermitidos = {"ADMINISTRADOR", "INMOBILIARIA", "CLIENTE"}; %>
<%@ include file="/WEB-INF/jspf/seguridad.jspf" %>
<%

    request.setCharacterEncoding("UTF-8");
    String accion = request.getParameter("accion");
    String destino = ctx + "/propiedad/listar.jsp";
    Connection con = null; PreparedStatement ps = null; ResultSet rs = null;

    boolean esGestor = "ADMINISTRADOR".equals(rolSesion) || "INMOBILIARIA".equals(rolSesion);

    try {
        con = abrirConexion();
        con.setAutoCommit(false);

        // Segunda validacion de rol, DENTRO del controlador (ademas de la pagina q
ue llama)
        if (("crear".equals(accion) || "editar".equals(accion) || "baja".equals(accion)) && !esGestor) {
            deshacer(con);
            response.sendRedirect(ctx + "/inicio.jsp?error=permiso");
            return;
        }
    }
```

Fíjate en rolesPermitidos = {"ADMINISTRADOR", "INMOBILIARIA", "CLIENTE"} — los tres, porque el CLIENTE también usa este mismo controlador para marcar favoritos. Por eso el if de abajo revalida específicamente que solo un gestor pueda crear/editar/dar de baja, aunque seguridad.jspf ya deje pasar a los tres roles a la página.

Acción crear (con transacción propiedad + características)

propiedad/acciones.jsp (parte 2: crear)

```
if ("crear".equals(accion)) {
    String matricula = request.getParameter("matricula");
    String titulo = request.getParameter("titulo");
    String descripcion = request.getParameter("descripcion");
    double precio = aDoble(request.getParameter("precio"), 0);
    int idCiudad = aEntero(request.getParameter("id_ciudad"), 0);
    int idTipo = aEntero(request.getParameter("id_tipo_propiedad"), 0);
    String[] caracteristicas = request.getParameterValues("caracteristicas");

    ps = con.prepareStatement(
        "INSERT INTO propiedad (matricula_inmobiliaria, titulo, descripcion, "
        + " precio, id_ciudad, id_tipo_propiedad, id_usuario) VALUES (?, ?, ?, ?, "
        + " ?, ?, ?)",
        Statement.RETURN_GENERATED_KEYS);
    ps.setString(1, matricula.trim());
    ps.setString(2, titulo.trim());
    ps.setString(3, descripcion);
    ps.setDouble(4, precio);
    ps.setInt(5, idCiudad);
    ps.setInt(6, idTipo);
    ps.setInt(7, idUsuarioSesion);
    ps.executeUpdate();

    rs = ps.getGeneratedKeys();
    rs.next();
    int idPropiedad = rs.getInt(1);
    cerrar(rs, ps);

    if (caracteristicas != null) {
        ps = con.prepareStatement(
            "INSERT INTO propiedad_caracteristica (id_propiedad, id_caracteristica) VALUES (?, ?)");
        for (String idCar : caracteristicas) {
            ps.setInt(1, idPropiedad);
            ps.setInt(2, aEntero(idCar, 0));
            ps.addBatch();
        }
        ps.executeBatch();
        cerrar(ps);
    }
    con.commit();
    destino += "?msg=" + java.net.URLEncoder.encode("Propiedad publicada correctamente.", "UTF-8");
}
```

Importante: Statement.RETURN_GENERATED_KEYS es lo que permite recuperar el id_propiedad recién creado con getGeneratedKeys(). Sin eso, no sabríamos a qué propiedad asociar las características que vienen justo después — es el mismo mecanismo que usa registrar.jsp para encadenar el INSERT de usuario con el de perfil.

Acción editar

propiedad/acciones.jsp (parte 3: editar)

```
    } else if ("editar".equals(accion)) {
        int idPropiedad = aEntero(request.getParameter("id_propiedad"), 0);
        String titulo = request.getParameter("titulo");
        String descripcion = request.getParameter("descripcion");
        double precio = aDoble(request.getParameter("precio"), 0);
        int idCiudad = aEntero(request.getParameter("id_ciudad"), 0);
        int idTipo = aEntero(request.getParameter("id_tipo_propiedad"), 0);
        String[] caracteristicas = request.getParameterValues("caracteristicas");

        ps = con.prepareStatement(
            "UPDATE propiedad SET titulo=?, descripcion=?, precio=?, id_ciudad=?, id_
            tipo_propiedad=? "
            + "WHERE id_propiedad=?");
        ps.setString(1, titulo.trim());
        ps.setString(2, descripcion);
        ps.setDouble(3, precio);
        ps.setInt(4, idCiudad);
        ps.setInt(5, idTipo);
        ps.setInt(6, idPropiedad);
        ps.executeUpdate();
        cerrar(ps);

        // Limpia las caracteristicas anteriores y vuelve a insertar las seleccionada
s
        ps = con.prepareStatement("DELETE FROM propiedad_caracteristica WHERE id_prop
        iedad = ?");
        ps.setInt(1, idPropiedad);
        ps.executeUpdate();
        cerrar(ps);

        if (caracteristicas != null) {
            ps = con.prepareStatement(
                "INSERT INTO propiedad_caracteristica (id_propiedad, id_caracteristic
a) VALUES (?, ?)");
            for (String idCar : caracteristicas) {
                ps.setInt(1, idPropiedad);
                ps.setInt(2, aEntero(idCar, 0));
                ps.addBatch();
            }
            ps.executeBatch();
            cerrar(ps);
        }
        con.commit();
        destino = ctx + "/propiedad/detalle.jsp?id=" + idPropiedad
            + "&msg=" + java.net.URLEncoder.encode("Propiedad actualizada.", "UTF-8")
;
    }
```

Importante: para editar las características, no se hace un UPDATE (no tendría sentido para una tabla intermedia). Se **borran todas** las asociaciones anteriores de esa propiedad y se **vuelven a insertar** las que quedaron marcadas — es un patrón simple y confiable: "borrar y reconstruir" en vez de calcular cuáles agregar y cuáles quitar.

Acciones baja y favorito

propiedad/acciones.jsp (parte 4: baja y favorito)

```
    } else if ("baja".equals(accion)) {
        int idPropiedad = aEntero(request.getParameter("id_propiedad"), 0);
        ps = con.prepareStatement("UPDATE propiedad SET estado = 'INACTIVA' WHERE id_
propiedad = ?");
        ps.setInt(1, idPropiedad);
        ps.executeUpdate();
        cerrar(ps);
        con.commit();
        destino += "?msg=" + java.net.URLEncoder.encode("Propiedad dada de baja.", "U
TF-8");

    } else if ("favorito".equals(accion)) {
        int idPropiedad = aEntero(request.getParameter("id_propiedad"), 0);

        // Si ya es favorita, la quita; si no lo es, la agrega (toggle)
        ps = con.prepareStatement(
            "SELECT id_favorito FROM favorito WHERE id_usuario = ? AND id_propiedad =
?");
        ps.setInt(1, idUsuarioSesion);
        ps.setInt(2, idPropiedad);
        rs = ps.executeQuery();
        boolean existeFav = rs.next();
        cerrar(rs, ps);

        if (existeFav) {
            ps = con.prepareStatement("DELETE FROM favorito WHERE id_usuario = ? AND
id_propiedad = ?");
            ps.setInt(1, idUsuarioSesion);
            ps.setInt(2, idPropiedad);
            ps.executeUpdate();
            cerrar(ps);
            destino = ctx + "/propiedad/detalle.jsp?id=" + idPropiedad
                + "&msg=" + java.net.URLEncoder.encode("Quitado de favoritos.", "UTF-
8");
        } else {
            ps = con.prepareStatement("INSERT INTO favorito (id_usuario, id_propiedad
) VALUES (?, ?)");
            ps.setInt(1, idUsuarioSesion);
            ps.setInt(2, idPropiedad);
            ps.executeUpdate();
            cerrar(ps);
            destino = ctx + "/propiedad/detalle.jsp?id=" + idPropiedad
                + "&msg=" + java.net.URLEncoder.encode("Agregado a favoritos.", "UTF-
8");
        }
        con.commit();
    }
}
```

Importante: "baja" nunca usa DELETE — solo cambia estado a INACTIVA. Es la baja lógica que exige el parcial: la fila sigue en la base de datos con todo su historial, solo deja de aparecer en el catálogo.

Cierre: Manejo de la restricción UNIQUE

propiedad/acciones.jsp (parte 5: cierre)

```
    } catch (SQLIntegrityConstraintViolationException ex) {
        deshacer(con);
        String m = ex.getMessage();
        String amigable = m.contains("uq_propiedad_matricula")
            ? "Ya existe una propiedad con esa matricula inmobiliaria."
            : "Datos duplicados: " + m;
        destino = ctx + "/propiedad/nueva.jsp?err=" + java.net.URLEncoder.encode(
            amigable, "UTF-8");
    } catch (SQLException ex) {
        deshacer(con);
        destino += "?err=" + java.net.URLEncoder.encode(ex.getMessage(), "UTF-8")
;
    } finally {
        cerrar(rs, ps, con);
    }
    response.sendRedirect(destino);
%>
```

Importante: `SQLIntegrityConstraintViolationException` es la excepción **específica** que lanza JDBC cuando se viola una llave única o foránea. Capturarla aparte (antes que el catch `(SQLException ex)` genérico) es lo que permite mostrar "Ya existe una propiedad con esa matrícula" en vez del mensaje técnico del motor, que el usuario final no entendería. Este mismo patrón se repite en `registrar.jsp`, `cita/acciones.jsp` y `solicitud/acciones.jsp`.

Paso 11. Módulo de citas

Paso 11.1 cita/nueva.jsp

cita/nueva.jsp

```
<% String[] rolesPermitidos = {"CLIENTE"}; %>
<%@ include file="/WEB-INF/jspf/seguridad.jspf" %>
<%
    int idPropiedad = aEntero(request.getParameter("id"), 0);
    // SELECT titulo FROM propiedad WHERE id_propiedad = ? (para mostrar el
    nombre)
%>
<form method="post" action="<%= ctx %>/cita/acciones.jsp">
    <input type="hidden" name="accion" value="crear">
    <input type="hidden" name="id_propiedad" value="<%= idPropiedad %>">
    <input type="datetime-local" name="fecha_hora" required>
    <textarea name="observaciones" maxlength="255"></textarea>
    <button type="submit">Agendar</button>
</form>
```

Importante: `fecha_hora.replace("T", " ")` — el input `type="datetime-local"` del navegador entrega el valor en formato ISO con una "T" en medio (ej: 2026-09-20T15:00), y hay que ajustarlo al formato DATETIME que espera MySQL. Y sobre `uq_cita_propiedad_fecha`: es la restricción UNIQUE que impide dos citas en la misma propiedad al mismo horario — la **consulta/restricción más importante de este módulo** para tu sustentación.

Paso 11.2 cita/acciones.jsp

cita/acciones.jsp

```
<% String[] rolesPermitidos = {"CLIENTE", "INMOBILIARIA", "ADMINISTRADOR"}; %>
<%@ include file="/WEB-INF/jspf/seguridad.jspf" %>
<%
    boolean esGestor = "ADMINISTRADOR".equals(rolSesion) || "INMOBILIARIA".equals(rolSesion);
    con.setAutoCommit(false);

    if ("crear".equals(accion)) {
        String fechaHora = request.getParameter("fecha_hora").replace("T", " ") + ":00";
        ps = con.prepareStatement(
            "INSERT INTO cita (id_propiedad, id_cliente, fecha_hora, observaciones) VALUES (?, ?, ?, ?)");
        ps.setInt(1, aEntero(request.getParameter("id_propiedad"), 0));
        ps.setInt(2, idUsuarioSesion);
        ps.setString(3, fechaHora);
        ps.setString(4, request.getParameter("observaciones"));
        ps.executeUpdate();
        con.commit();

    } else if ("cancelar".equals(accion)) {
        int idCita = aEntero(request.getParameter("id_cita"), 0);
        if ("CLIENTE".equals(rolSesion)) {
            // El cliente SOLO cancela sus propias citas
            ps = con.prepareStatement("UPDATE cita SET estado='CANCELADA' WHERE id_cita=? AND id_cliente=?");
            ps.setInt(1, idCita); ps.setInt(2, idUsuarioSesion);
        } else {
            ps = con.prepareStatement("UPDATE cita SET estado='CANCELADA' WHERE id_cita=?");
            ps.setInt(1, idCita);
        }
        ps.executeUpdate();
        con.commit();

    } else if ("confirmar".equals(accion) && esGestor) {
        ps = con.prepareStatement("UPDATE cita SET estado='CONFIRMADA' WHERE id_cita=?");
        ps.setInt(1, aEntero(request.getParameter("id_cita"), 0));
        ps.executeUpdate();
        con.commit();

    } else if ("realizada".equals(accion) && esGestor) {
        ps = con.prepareStatement("UPDATE cita SET estado='REALIZADA' WHERE id_cita=?");
        ps.setInt(1, aEntero(request.getParameter("id_cita"), 0));
        ps.executeUpdate();
        con.commit();
    }
} catch (SQLIntegrityConstraintViolationException ex) {
    deshacer(con);
    String amigable = ex.getMessage().contains("uq_cita_propiedad_fecha")
        ? "Esa propiedad ya tiene una cita agendada en ese horario. Elige otra fecha/hora."
        : "Datos duplicados.";
    destino = ctx + "/cita/nueva.jsp?id=" + request.getParameter("id_propiedad")
        + "&err=" + java.net.URLEncoder.encode(amigable, "UTF-8");
}
```

Paso 11.3 cita/mis-citas.jsp

cita/mis-citas.jsp

```
<% String[] rolesPermitidos = {"CLIENTE"}; %>
<%@ include file="/WEB-INF/jspf/seguridad.jspf" %>
<%
    ps = con.prepareStatement(
        "SELECT c.id_cita, c.fecha_hora, c.estado, p.titulo "
        + "FROM cita c JOIN propiedad p ON p.id_propiedad = c.id_propiedad "
        + "WHERE c.id_cliente = ? ORDER BY c.fecha_hora DESC");
    ps.setInt(1, idUsuarioSesion);
    rs = ps.executeQuery();
    // tabla con boton Cancelar si estado es PENDIENTE o CONFIRMADA
%>
```

Importante: esAdmin ? "" : "WHERE p.id_usuario = ?" es la línea que hace que un agente **solo vea las citas de sus propias propiedades**, mientras que el administrador ve todas — la misma condición se repite en solicitud/gestion.jsp.

Paso 11.4 cita/agenda.jsp

cita/agenda.jsp

```
<% String[] rolesPermitidos = {"INMOBILIARIA", "ADMINISTRADOR"}; %>
<%@ include file="/WEB-INF/jspf/seguridad.jspf" %>
<%
    boolean esAdmin = "ADMINISTRADOR".equals(rolSesion);
    String sql = "SELECT c.id_cita, c.fecha_hora, c.estado, p.titulo, u.username
AS cliente "
        + "FROM cita c "
        + " JOIN propiedad p ON p.id_propiedad = c.id_propiedad "
        + " JOIN usuario u ON u.id_usuario = c.id_cliente "
        + (esAdmin ? "" : "WHERE p.id_usuario = ? ")
        + "ORDER BY c.fecha_hora DESC";
    ps = con.prepareStatement(sql);
    if (!esAdmin) ps.setInt(1, idUsuarioSesion);
    rs = ps.executeQuery();
    // por fila: boton Confirmar si PENDIENTE, Marcar realizada si CONFIRMADA, Ca
ncelar siempre
%>
```

Paso 12. Módulo de solicitudes

Paso 12.1 solicitud/nueva.jsp

solicitud/nueva.jsp

```
<% String[] rolesPermitidos = {"CLIENTE"}; %>
<%@ include file="/WEB-INF/jspf/seguridad.jspf" %>
<%-- ... consulta el titulo de la propiedad, igual que cita/nueva.jsp ... --%>
<form method="post" action="<%= ctx %>/solicitud/acciones.jsp">
    <input type="hidden" name="accion" value="crear">
    <input type="hidden" name="id_propiedad" value="<%= idPropiedad %>">
    <select name="tipo">
        <option value="COMPRA">Compra</option>
        <option value="ARRIENDO">Arriendo</option>
    </select>
    <textarea name="observaciones" maxlength="255"></textarea>

    <!-- Documentos: pares repetidos de input+select con el mismo name -->
    <input name="nombreDoc" placeholder="cedula.pdf">
    <select name="tipoDoc"><option>CEDULA</option>...</select>
    <input name="nombreDoc" placeholder="certificado_laboral.pdf (opcional)">
    <select name="tipoDoc">...</select>

    <button type="submit">Radical solicitud</button>
</form>
```

Recordatorio de la decisión de alcance: aquí es donde se ve en código lo que ya documentamos — los documentos se radican por nombre y tipo (texto), no como archivo real subido, porque el manejo de multipart/form-data en JSP puro es bastante más complejo. Vale la pena repetir esta nota como "Importante" justo en este paso también, no solo en la introducción del módulo.

Paso 12.2 solicitud/acciones.jsp

Importante: la solicitud y sus documentos se guardan dentro de la **misma transacción** (mismo patrón que propiedad + características). El bucle for recorre los dos arreglos paralelos (nombresDoc[] y tiposDoc[]) a la vez, usando continue para saltarse cualquier campo de documento que quedó vacío, y executeBatch() para insertar todos de un solo viaje a la base de datos.

solicitud/acciones.jsp

```
<% String[] rolesPermitidos = {"CLIENTE", "INMOBILIARIA", "ADMINISTRADOR"}; %>
<%@ include file="/WEB-INF/jspf/seguridad.jspf" %>
<%
    con.setAutoCommit(false);

    if ("crear".equals(accion)) {
        ps = con.prepareStatement(
            "INSERT INTO solicitud (id_propiedad, id_cliente, tipo, observaciones) VALUES
            (?, ?, ?, ?)",
            Statement.RETURN_GENERATED_KEYS);
        ps.setInt(1, aEntero(request.getParameter("id_propiedad"), 0));
        ps.setInt(2, idUsuarioSesion);
        ps.setString(3, request.getParameter("tipo"));
        ps.setString(4, request.getParameter("observaciones"));
        ps.executeUpdate();
        rs = ps.getGeneratedKeys();
        rs.next();
        int idSolicitud = rs.getInt(1);
        cerrar(rs, ps);

        String[] nombresDoc = request.getParameterValues("nombreDoc");
        String[] tiposDoc = request.getParameterValues("tipoDoc");
        ps = con.prepareStatement(
            "INSERT INTO documento_solicitud (id_solicitud, nombre_archivo, tipo_documento)
            VALUES (?, ?, ?)");
        for (int i = 0; i < nombresDoc.length; i++) {
            if (nombresDoc[i] == null || nombresDoc[i].trim().isEmpty()) continue;
            ps.setInt(1, idSolicitud);
            ps.setString(2, nombresDoc[i].trim());
            ps.setString(3, tiposDoc[i]);
            ps.addBatch();
        }
        ps.executeBatch();
        con.commit();

    } else if (("aprobar".equals(accion) || "rechazar".equals(accion)) && esGestor) {
        String nuevoEstado = "aprobar".equals(accion) ? "APROBADA" : "RECHAZADA";
        ps = con.prepareStatement("UPDATE solicitud SET estado = ? WHERE id_solicitud = ?
        ");
        ps.setString(1, nuevoEstado);
        ps.setInt(2, aEntero(request.getParameter("id_solicitud"), 0));
        ps.executeUpdate();
        con.commit();
    }
}
```

Importante: la solicitud y sus documentos se guardan dentro de la **misma transacción** (mismo patrón que propiedad + características). El bucle for recorre los dos arreglos paralelos (nombresDoc[] y tiposDoc[]) a la vez, usando continue para saltarse cualquier campo de

documento que quedó vacío, y executeBatch() para insertar todos de un solo viaje a la base de datos.

Paso 12.3 solicitud/mis-solicitudes.jsp

solicitud/mis-solicitudes.jsp

```
<% String[] rolesPermitidos = {"CLIENTE"}; %>
<%@ include file="/WEB-INF/jspf/seguridad.jspf" %>
<%
    ps = con.prepareStatement(
        "SELECT s.id_solicitud, s.tipo, s.estado, s.fecha_solicitud, p.titulo "
        + "FROM solicitud s JOIN propiedad p ON p.id_propiedad = s.id_propiedad "
        + "WHERE s.id_cliente = ? ORDER BY s.fecha_solicitud DESC");
    ps.setInt(1, idUsuarioSesion);
    rs = ps.executeQuery();
    while (rs.next()) {
        int idSolicitud = rs.getInt("id_solicitud");
        // Por cada solicitud, una sub-consulta a documento_solicitud:
        // SELECT nombre_archivo, tipo_documento FROM documento_solicitud WHERE i
d_solicitud = ?
        // ... se lista cada documento debajo de su solicitud ...
    }
%>
```

Paso 12.4 solicitud/gestión.jsp

solicitud/gestion.jsp

```
<% String[] rolesPermitidos = {"INMOBILIARIA", "ADMINISTRADOR"}; %>
<%@ include file="/WEB-INF/jspf/seguridad.jspf" %>
<%
    boolean esAdmin = "ADMINISTRADOR".equals(rolSesion);
    String sql = "SELECT s.id_solicitud, s.tipo, s.estado, p.titulo, u.username A
S cliente "
        + "FROM solicitud s "
        + " JOIN propiedad p ON p.id_propiedad = s.id_propiedad "
        + " JOIN usuario u ON u.id_usuario = s.id_cliente "
        + (esAdmin ? "" : "WHERE p.id_usuario = ? ")
        + "ORDER BY s.fecha_solicitud DESC";
    ps = con.prepareStatement(sql);
    if (!esAdmin) ps.setInt(1, idUsuarioSesion);
    rs = ps.executeQuery();
    // botones Aprobar/Rechazar solo si estado == PENDIENTE
%>
```

Paso 13. Administración de usuarios

Paso 13.1 admin/usuarios.jsp

admin/usuarios.jsp

```
<% String[] rolesPermitidos = {"ADMINISTRADOR"}; %>
<%@ include file="/WEB-INF/jspf/seguridad.jspf" %>
<%
    ps = con.prepareStatement(
        "SELECT u.id_usuario, u.username, u.correo, u.activo, r.nombre AS rol, u.id_rol "
        + "FROM usuario u JOIN rol r ON r.id_rol = u.id_rol ORDER BY u.username");
    rs = ps.executeQuery();
    while (rs.next()) {
        int idUsuario = rs.getInt("id_usuario");
%>
        <form method="post" action="<%= ctx %>/admin/acciones.jsp">
            <input type="hidden" name="accion" value="cambiarRol">
            <input type="hidden" name="id_usuario" value="<%= idUsuario %>">
            <select name="id_rol" <%= idUsuario == idUsuarioSesion ? "disabled" : "" %>
        >
                <!-- opciones generadas desde SELECT * FROM rol, con la actual "selecte
d" --%>
            </select>
            <% if (idUsuario != idUsuarioSesion) { %>
                <button type="submit">Guardar</button>
            <% } %>
        </form>

        <form method="post" action="<%= ctx %>/admin/acciones.jsp">
            <input type="hidden" name="accion" value="toggleActivo">
            <input type="hidden" name="id_usuario" value="<%= idUsuario %>">
            <input type="hidden" name="valor" value="<%= rs.getBoolean("activo") ? 0 :
1 %>">
            <% if (idUsuario != idUsuarioSesion) { %>
                <button type="submit"><%= rs.getBoolean("activo") ? "Inactivar" : "Acti
var" %></button>
            <% } else { %>
                (tu)
            <% } %>
        </form>
    <% } %>
```

Importante: `idUsuario == idUsuarioSesion ? "disabled" : ""` bloquea en la propia vista que un administrador se cambie su rol a sí mismo (o lo desactive) — pero como toda validación de vista, es solo la primera barrera. La segunda, la que realmente importa, está en el controlador.

Paso 13.2 admin/acciones.jsp

admin/acciones.jsp

```
<% String[] rolesPermitidos = {"ADMINISTRADOR"}; %>
<%@ include file="/WEB-INF/jspf/seguridad.jspf" %>
<%
    int idUsuario = aEntero(request.getParameter("id_usuario"), 0);

    // Un administrador no puede modificarse a si mismo (evita quedarse sin acceso)
    if (idUsuario == idUsuarioSesion) {
        response.sendRedirect(ctx + "/admin/usuarios.jsp");
        return;
    }

    con = abrirConexion();
    if ("cambiarRol".equals(accion)) {
        ps = con.prepareStatement("UPDATE usuario SET id_rol = ? WHERE id_usuario = ?");
        ps.setInt(1, aEntero(request.getParameter("id_rol"), 0));
        ps.setInt(2, idUsuario);
        ps.executeUpdate();
    } else if ("toggleActivo".equals(accion)) {
        ps = con.prepareStatement("UPDATE usuario SET activo = ? WHERE id_usuario = ?");
        ps.setInt(1, aEntero(request.getParameter("valor"), 1));
        ps.setInt(2, idUsuario);
        ps.executeUpdate();
    }
}
```

Importante: este es el ejemplo más claro de "doble candado" de todo el proyecto. La vista ya deshabilita el `<select>` para el propio admin, pero **eso solo afecta lo que se ve en el navegador** — cualquiera podría manipular el HTML o mandar la petición directamente. La verificación real, la que de verdad importa, es la comparación `idUsuario == idUsuarioSesion` en el servidor: si alguien intenta saltarse la vista, el controlador igual lo rechaza.

Paso 14. Reportes

Pantalla exclusiva del administrador con las tres consultas que completan las 5 obligatorias del parcial (la primera INNER JOIN y la resolución de N:M ya están en listar.jsp y detalle.jsp, respectivamente).

reportes.jsp

```
<% String[] rolesPermitidos = {"ADMINISTRADOR"}; %>
<%@ include file="/WEB-INF/jspf/seguridad.jspf" %>

<!-- Reporte 1: agregacion (consulta obligatoria #5) --%>
<%
    rs = st.executeQuery(
        "SELECT c.nombre AS ciudad, p.estado, COUNT(*) AS cantidad "
        + "FROM propiedad p JOIN ciudad c ON c.id_ciudad = p.id_ciudad "
        + "GROUP BY c.nombre, p.estado HAVING COUNT(*) >= 1 ORDER BY cantidad DESC");
%>

<!-- Reporte 2: LEFT JOIN (consulta obligatoria #4) --%>
<%
    rs = st.executeQuery(
        "SELECT p.matricula_inmobiliaria, p.titulo, p.precio "
        + "FROM propiedad p LEFT JOIN cita c ON c.id_propiedad = p.id_propiedad "
        + "WHERE c.id_cita IS NULL AND p.estado <> 'INACTIVA'");
%>

<!-- Reporte 3: INNER JOIN de 4 tablas (consulta obligatoria #2) --%>
<%
    rs = st.executeQuery(
        "SELECT ci.nombre AS ciudad, p.titulo, u.username AS cliente, c.fecha_hora,
        c.estado "
        + "FROM cita c "
        + "  JOIN propiedad p ON p.id_propiedad = c.id_propiedad "
        + "  JOIN ciudad ci ON ci.id_ciudad = p.id_ciudad "
        + "  JOIN usuario u ON u.id_usuario = c.id_cliente "
        + "ORDER BY c.fecha_hora");
%>
```

Reporte	Qué resuelve
1 (GROUP BY/HAVING)	Cuenta propiedades agrupadas por ciudad y estado — alimenta la historia #12.
2 (LEFT JOIN)	Propiedades que no tienen ninguna cita (IS NULL detecta la ausencia de coincidencia).

3 (INNER JOIN de 4 tablas)	Segunda consulta de unión múltiple del proyecto, distinta a la del listado de propiedades.
----------------------------	--

Paso 15. Desplegar y probar la aplicación

Paso 15.1 Puesta en marcha

#	Acción	Resultado esperado
1	Iniciar MySQL	El servicio aparece en ejecución.
2	Ejecutar 01_base_datos_inmobiliaria_completo.sql.	13 tablas y 10 usuarios creados.
3	Copiar el driver JDBC a WEB-INF/lib.	El .jar se ve dentro de la carpeta.
4	Ajustar DB_USUARIO/DB_CLAVE en conexion.jspf.	Coinciden con tu instalación de MySQL.
5	Ejecutar el proyecto en Tomcat.	Se abre http://localhost:8080/InmobiliariaWeb/ y muestra la landing.
6	Registrar una cuenta nueva de prueba.	Confirma que el flujo público funciona antes de entrar con los usuarios precargados.

Paso 15.2 Guion de prueba completa

#	Usuario	Acción	Qué debe ocurrir
1	(Nadie)	Entrar a /	Landing pública, buscador y propiedades destacadas visibles sin sesión.
2	(Nadie)	Ver detalle de una propiedad	Ficha completa con galería, pero sin botones de agendar/solicitar — solo invitación a iniciar sesión.
3	(Nadie)	Registrarse	Cuenta creada con rol CLIENTE; redirige al login.
4	Cliente1	Buscar con filtros (ciudad, tipo, precio)	El listado se reduce según lo filtrado.
5	Cliente1	Marcar una propiedad como favorita	Mensaje de confirmación; repetir la acción la quita.

6	Cliente1	Agendar una cita	Se crea con estado PENDIENTE.
7	Cliente1	Repetir la misma cita (misma propiedad y horario)	Mensaje: "Esa propiedad ya tiene una cita agendada en ese horario."
8	Cliente1	Radicar una solicitud con documentos	Aparece en "Mis solicitudes" con estado PENDIENTE.
9	Agente1 (Dueño de la propiedad)	Confirmar la cita desde Agenda	Confirmar la cita desde Agenda
10	Agente1	Aprobar la solicitud desde Solicitudes	Estado pasa a APROBADA.
11	Agente1	Publicar una propiedad con matrícula repetida	Mensaje: "Ya existe una propiedad con esa matrícula inmobiliaria."
12	Admin	Ver 'Reportes'	Los 3 reportes muestran datos coherentes con lo hecho hasta ahora.
13	Admin	Cambiar el rol de cliente1 a INMOBILIARIA	Al volver a iniciar sesión, cliente1 ve el menú de agente.
14	Cliente1 (Sesión activa)	Escribir la URL admin/usuarios.jsp directamente	Redirige a inicio.jsp con aviso de "sin permisos" — no lo deja entrar.

Paso 15.3 Comprobación en la base de datos

Después del recorrido, ejecuta esto para confirmar que la baja lógica y las transacciones dejaron todo consistente:

```
-- La propiedad dada de baja sigue existiendo, solo cambia su estado
SELECT id_propiedad, titulo, estado FROM propiedad WHERE estado = 'INACTIVA';

-- Historial de una cita: quien la agendo, quien la confirmo
SELECT c.fecha_hora, c.estado, u.username AS cliente
FROM cita c JOIN usuario u ON u.id_usuario = c.id_cliente
ORDER BY c.fecha_hora DESC;
```

16. Consultas obligatorias

#	Consulta	Ubicación	Tablas involucradas
1	INNER JOIN de 3+ tablas	propiedad/listar.jsp	propiedad + ciudad + tipo_propiedad + usuario
2	INNER JOIN de 3+ tablas (segunda)	reportes.jsp	cita + propiedad + ciudad + usuario
3	Resuelve relación N:M	propiedad/detalle.jsp	propiedad_caracteristica + caracteristica
4	LEFT JOIN	reportes.jsp	propiedad LEFT JOIN cita
5	Agregación GROUP BY/HAVING	reportes.jsp	propiedad + ciudad, agrupado por ciudad y estado

17. Metodología Scrum (Resumen)

El desarrollo se organizó en 3 sprints de 7 días, con Product Backlog, Sprint Planning, Sprint Review y Sprint Retrospective documentados en archivos .md separados dentro de la carpeta scrum/ del repositorio (no repetidos aquí para no duplicar contenido — el documento completo de cada sprint vive en scrum/product-backlog.md, sprint1.md, sprint2.md, sprint3.md).

Sprint	Contenido
Sprint 1 — Cimientos y acceso	MER, DDL/DML, conexión JDBC, landing, registro, login/roles
Sprint 2 — Núcleo del negocio	CRUD de propiedades con imágenes y características, filtros, perfil, favoritos
Sprint 3 — Operación y cierre	Citas, solicitudes, reportes con agregación, pruebas unitarias, documentación final

El tablero de seguimiento (GitHub Projects) y el historial de commits están disponibles en el repositorio público.

18. Pruebas unitarias

Como el proyecto no usa Maven/Gradle (todo va en JSP/JSPF puro, según las reglas del curso), la lógica crítica de utilidades.jspf se extrajo a una clase Java independiente (pruebas/Utilidades.java) para poder probarla sin necesitar un servidor Tomcat corriendo. Se

escribieron 13 pruebas (pruebas/PruebasUnitarias.java) sobre el cifrado de contraseñas, la conversión segura de parámetros, y el escape de HTML. **Resultado: 13/13 pruebas exitosas.**

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\xampp\tomcat\webapps\InmobiliariaWeb\pruebas> javac Utilidades.java
PS C:\xampp\tomcat\webapps\InmobiliariaWeb\pruebas> javac PruebasUnitarias.java
PS C:\xampp\tomcat\webapps\InmobiliariaWeb\pruebas> javac Utilidades.java PruebasUnitarias.java java PruebasUnitarias
error: Class names, 'java,PruebasUnitarias', are only accepted if annotation processing is explicitly requested
1 error
PS C:\xampp\tomcat\webapps\InmobiliariaWeb\pruebas> java PruebasUnitarias
=== Pruebas unitarias: Utilidades ===

[OK] claveCifrada es determinista (mismo usuario+clave -> mismo hash)
[OK] claveCifrada distingue usuarios distintos con la misma clave
[OK] claveCifrada siempre produce un hash de 64 caracteres (SHA-256)
[OK] claveCifrada('admin','1234') coincide con el hash conocido de la BD
[OK] aEntero convierte un texto numerico valido
[OK] aEntero devuelve el valor por defecto si el texto no es numerico
[OK] aEntero devuelve el valor por defecto si el texto es null
[OK] aDoble convierte un texto con punto decimal
[OK] aDoble convierte un texto con coma decimal (formato local)
[OK] aDoble devuelve el valor por defecto si el texto no es numerico
[OK] esc escapa las etiquetas HTML (previene XSS)
[OK] esc no falla con texto null
[OK] pesos formatea correctamente un valor entero (mismos digitos, con simbolo $)

=== Resultado: 13/13 pruebas exitosas ===
PS C:\xampp\tomcat\webapps\InmobiliariaWeb\pruebas> |
```

19. Errores frecuentes y cómo resolverlos

Esta tabla consolida los problemas reales que encontraste durante el desarrollo.

Mensaje o síntoma	Causa	Solución
WinError 32: el proceso no tiene acceso al archivo, al ejecutar el script SQL desde "Run SQL Script"	Bloqueo de archivo temporal (sincronización de OneDrive con el Escritorio)	Ejecutar el script directo desde el editor de Workbench (Ctrl+A + Ctrl+Shift+Enter) en vez del diálogo de archivo
"Usuario o clave incorrectos" pese a que el hash calculado y el guardado coincidían al probarlos por separado	Corrupción de caracteres al copiar y pegar acceso.jsp	Se aisló el problema con páginas de diagnóstico temporales (comparando el hash calculado contra el guardado); se reemplazó el archivo completo
Tag [<%] no terminado en listar.jsp / formularios sin botón visible	Corrupción similar al copiar y pegar código (recurrente)	Práctica establecida: ante cualquier comportamiento raro sin causa obvia, borrar y

		volver a pegar el archivo completo en vez de depurar línea por línea
El mensaje de matrícula duplicada no se mostraba en pantalla	nueva.jsp no leía el parámetro err de la URL de redirección	Se agregó la lectura y renderizado del parámetro en la vista
Comandos de PowerShell fallando al pegarse varias líneas juntas	Comandos de PowerShell fallando al pegarse varias líneas juntas	Ejecutar los comandos de uno en uno, con Enter después de cada línea, en vez de pegar bloques completos
Historia "Administrador asigna y revoca roles" documentada como completada en el Sprint 1, pero nunca implementada	No se verificó la documentación contra el código real al cerrar el sprint	Se detectó al construir el diagrama de casos de uso; se implementó admin/usuarios.jsp antes de cerrar la documentación final

20. Conclusiones

El proyecto cubre el 100% de los criterios de evaluación del parcial: aplicación web funcional con control de acceso por rol validado en el servidor (login cifrado, paneles diferenciados, CRUD de propiedades, citas y solicitudes en operación), metodología Scrum documentada en tres sprints con tablero de seguimiento e historial de commits, y un modelo de datos normalizado con las tres relaciones exigidas (1:1, 1:N, N:M), restricciones UNIQUE, script DDL/DML con datos de prueba, las 5 consultas obligatorias, y pruebas unitarias sobre la lógica crítica.

El desarrollo bajo presión de tiempo obligó a tomar decisiones de alcance explícitas — no implementar la carga binaria real de documentos en las solicitudes, y no construir el historial de auditoría (historia de prioridad Baja, marcada como valor agregado opcional en el propio enunciado) — documentadas en el punto exacto en que se tomaron, en vez de omitirlas en silencio. También se detectó y corrigió a tiempo una inconsistencia real entre la documentación Scrum y el código (la gestión de roles, documentada como completada sin haberse construido), gracias a la revisión sistemática hecha para este mismo documento — lo cual demuestra que documentar con cuidado no es un trámite aparte del desarrollo, sino una forma más de encontrar errores.

Anexo A. Inventario de archivos entregados

Archivo/Carpeta	Contenido
-----------------	-----------

01_base_datos_inmobiliaria_completo.sql	DDL de las 13 tablas + datos de prueba (10 usuarios, 10 propiedades, 10 citas, 10 solicitudes, entre otros)
diccionario_datos.md	Diccionario de datos completo, campo por campo
MER_inmobiliaria_completo.png	Diagrama entidad-relación
casos_de_uso.png	Diagrama de casos de uso UML
WEB-INF/jspf/ (6 archivos)	conexion, utilidades, seguridad, sesionOpcional, cabecera, pie
Raíz (9 archivos)	index, login, acceso, logout, registro, registrar, landing, inicio, reportes
propiedad/ (5 archivos)	listar, nueva, editar, detalle, acciones
cita/ (4 archivos)	nueva, acciones, mis-citas, agenda
solicitud/ (4 archivos)	nueva, acciones, mis-solicitudes, gestion
admin/ (2 archivos)	usuarios, acciones
css/estilos.css	Estilos propios sobre Bootstrap 5
scrum/ (4 archivos)	product-backlog, sprint1, sprint2, sprint3
pruebas/ (2 archivos)	Utilidades.java, PruebasUnitarias.java
Evidencias/	Capturas de pantalla de las pruebas de funcionamiento
Documento_Final_Inmobiliaria_Web.pdf	Documento_Final_Inmobiliaria_Web.pdf